GENERATIVE MODELS VS TRADITIONAL AI-DRIVEN ANOMALY DETECTION IN LARGE-SCALE SYSTEM LOGS – A COMPARATIVE STUDY



by

Muhammad Asad Mahmood Khan

M.S., Artificial Intelligence, Air University, 2026

A thesis submitted in partial fulfillment of the requirements for the degree of

Master of Science in Artificial Intelligence

AIR UNIVERSITY

2026

GENERATIVE MODELS VS TRADITIONAL AI-DRIVEN ANOMALY DETECTION IN LARGE-SCALE SYSTEM LOGS – A COMPARATIVE STUDY



by

Muhammad Asad Mahmood Khan

A thesis submitted in partial fulfillment of the requirements for the degree of

Master of Science in Artificial Intelligence

SUPERVISOR

Prof. Dr. Imran Ihsan

AIR UNIVERSITY

ISLAMABAD

© [Muhammad Asad Mahmood Khan] [2026]. All rights reserved.

CERTIFICATE

Department of Creative Technologies

It is hereby certified that Muhammad Asad Mahmood Khan (243260) has successfully completed his thesis.

Dr. Imran Ihsan

Professor

Air University

Supervisor

Dr. Tahir Akram

Associate Professor

Air University

Internal Examiner

Guidance and Evaluation Committee

Dr. _____

Associate Professor

_____*University*

External Examiner

Guidance and Evaluation Committee

Dr. Imran Ihsan

Associate Professor

Air University

Chair Department of Creative Technology

Dr. Mehdi Hassan

Professor

Air University

Dean Faculty of Creative Technology

Dr. Mehdi Hassan

Air University

Dean Faculty of Graduate Studies

DECLARATION

I declare that all material in this thesis is my work, and that which is not my own work has been mentioned as such, and that no material from this work has previously been submitted or approved for the award of a degree by this or any other university.

Signature: _____

Author's Name: Muhammad Asad Mahmood Khan

Dated: _____

It is certified that the work presented in this thesis is carried out and completed under my supervision.

Signature: _____

Supervisor's Name: **Dr. Imran Ihsan**

Professor

Department of Creative Technology

Air University, Islamabad

Dated: _____

Acknowledgments

Firstly, I am thankful to Allah Almighty for giving me the courage and strength to complete my Masters. Also, I would like to thank my supervisor Dr. Imran Ihsan for their essential advice and assistance throughout my master's programme. They gave me guidance and encouragement as I completed this research and prepared this thesis.

I also want to express my gratitude to my extended family and friends for their love and support throughout this process.

I would give my parents credit for this and all of my accomplishments. They gave me every chance to develop and did everything to support my development into a confident, intelligent man.

Generative Models Vs Traditional Ai-Driven Anomaly Detection In Large-Scale System Logs – A Comparative Study

by

Muhammad Asad Mahmood Khan

M.S., Artificial Intelligence, Air University, 2026

Submitted in partial fulfillment of the requirements for the degree of

Master of Science in Artificial Intelligence, Air University

Abstract

The escalating scale and complexity of distributed computing architectures, specifically Hadoop (HDFS), Apache Spark, and OpenStack, have rendered manual and rule-based log monitoring fundamentally intractable. This research presents a comprehensive comparative study evaluating Traditional Machine Learning (ML), sequence-based Deep Learning (DL), and Generative AI models for automated anomaly detection within massive, unstructured log collections. (Zhu, 2023)

Initial benchmarking demonstrates a critical functional dichotomy in current industry standards. Traditional supervised algorithms achieve high precision on structured data but remain brittle, relying heavily on labor-intensive manual feature engineering and labeled datasets. Furthermore, traditional unsupervised baselines, like Isolation Forest, exhibit significantly lower efficacy when processing high-dimensional sequences, exposing a severe gap in identifying unknown, zero-day errors.

To transcend these limitations, this thesis rigorously evaluates deep sequence and Generative AI architectures (including LSTMs, VAEs, and GANs). To ensure accurate assessment across heterogeneous environments, models are independently trained and tested within the same respective dataset boundaries for HDFS, Spark, and Hadoop, establishing true baselines without cross-system contamination.

This study delineates the operational boundaries of these paradigms. Findings reveal that while supervised Deep Learning eliminates manual feature extraction, Generative AI models offer theoretical automation advantages via a normal-only training paradigm but consistently underperform supervised approaches across all evaluated datasets.

Ultimately, this research empirically demonstrates that Traditional ML and supervised Deep Learning models outperform Generative architectures in log anomaly detection, while highlighting key precision trade-offs in concurrent environments like Spark.
Thesis Supervisor: Prof. Dr. Imran Ihsan

Table of Contents

Chapter 1: Introduction 13

1.1 System Logs 13

Why Are System Logs Generated? 14

1.1.1 Troubleshooting 14

1.1.2 Security Monitoring 14

1.1.3 System and Network Monitoring 14

1.1.4 Auditing 14

1.1.5 Incident Investigation 14

1.2 Types of System Logs 15

1.2.1 Authentication Logs 15

1.2.2 Application Logs 15

1.2.3 Operating-System Logs 15

1.2.4 Security Logs 15

1.2.5 Network Logs 15

1.3 Importance of Log Analysis 16

1.4 Anomalies in System Logs 16

1.5 Types of Log Anomalies 17

Point Anomalies 17

1.5.1 Contextual Anomalies 17

1.5.2 Collective Anomalies 17

1.6 Challenges in Log Anomaly Detection 17

1.6.1 Large Volume of Data 17

1.6.2 Diverse Log Formats....17
1.6.3 Sequential Dependencies....18
1.6.4 Changing System Behavior....18
1.6.5 Unknown Anomalies....18
1.7 Artificial Intelligence for Log Anomaly Detection....18
1.8 Motivation of the Study....19
1.9 Importance of Log Anomaly Detection....19
1.9.1 Security and Early Warning....19
1.9.2 Detection of System Failures....20
1.9.3 Reduction of Manual Log Monitoring....21
1.9.4 Faster Incident Response....21
1.9.5 Support for Incident Investigation and Auditing....21
1.9.6 Detection of Sequential and Collective Anomalies....22
Importance for Intelligent Log Analysis....22
1.9.7 Overall Importance....23
1.9.8 System Logs in Distributed Architectures: Hadoop, HDFS, and Apache Spark....23
1.10 Introduction to Distributed System Observability....23
1.11 The Hadoop Ecosystem and Resource Management Logs....24
1.12 HDFS Logs: The Storage Layer....24
1.13 Apache Spark Logs: The Computational Layer....25
1.14 Rationale for Inclusion in Anomaly Detection Research....26
1.15 Problem Statement....27
1.16 Motivation....28
1.17 Objectives....29
1.18 Scope of the thesis....29
1.19 Thesis Layout....30
Chapter 2: Literature Survey....32

2.1 Introduction to the Literature Review 32
2.2 System Logs and Their Role in Anomaly Detection 33
2.3 Types of Log Anomalies 34
2.4 Traditional Rule-Based Monitoring 34
2.5 Traditional Machine Learning: PCA and Statistical Methods 35
2.6 Traditional Machine Learning: Classification and Clustering 36
2.7 Isolation Forest and One-Class Learning 37
2.8 Log Parsing and Feature Engineering 38
2.9 Deep Learning for Log Analysis 39
2.10 DeepLog: Sequence Prediction 39
2.11 LogAnomaly and Semantic Sequence Modeling 40
2.12 LogRobust and Unstable Logs 41
2.13 CNN-Based Log Anomaly Detection 41
2.14 Transformers and Attention Mechanisms 42
2.15 Hit Anomaly: Modelling Templates and Parameters 43
2.16 LogBERT and Self-Supervised Learning 43
2.17 Generative Models for Anomaly Detection 44
2.18 LogGAN and Adversarial Learning 45
2.19 Autoencoders and VAE-Based Detection 45
2.20 Datasets Used in Log Anomaly Research 46
2.21 HDFS and Distributed File-System Logs 47
2.22 BGL, Hadoop, Spark, and OpenStack 48
2.23 Evaluation Metrics 48
2.24 Computational and Deployment Considerations 49
2.25 Robustness to Log Evolution and OOV Events 50
2.26 Normal-Only Training and Zero-Day Detection 50
2.27 Research Gap 51

2.28 Comparative Analysis of Existing Approaches 52
2.29 Proposed Experimental Direction 53
2.30 Literature Review & Preliminary Benchmarking 54
2.31 Foundations 57
2.31.1 Books 58
2.31.2 Papers 59
2.31.3 Experiments 59
2.32 Prior Work 60
2.33 Summary of the Literature Review 61
Chapter 3: Methodology 63
3.1 Research Design and Overall Methodology 64
3.2 Phase 1: Multi-Source Dataset Collection 65
3.2.1 HDFS Dataset 65
3.2.2 Apache Spark Dataset 65
3.2.3 Hadoop/OpenStack Dataset 65
Dataset Challenges and Mitigation Strategy 66
3.2.4 Apache Spark: Memory and Scale 66
3.2.5 Extreme Class Imbalance in Spark 66
3.2.6 Compute Limits 66
3.2.7 Hadoop/OpenStack: Reverse Class Imbalance 67
3.2.8 Traditional One-Class SVM Limitation 67
3.3 Phase 2: Log Preprocessing and Parsing 67
3.3.1 Template Extraction 68
3.3.2 Event ID Extraction 68
3.3.3 Timestamp Normalization 68
3.4 Phase 3: Comparative Model Architectures 68
3.4.1 Pipeline 1: Traditional Machine Learning 68

3.4.2 Event-Count Features....69
3.4.3 Deep Learning Pipeline: Sequence Embedding....69
3.4.4 Bidirectional Bi-LSTM....69
3.4.5 Attention Mechanisms....69
3.4.6 Generative AI Pipeline....70
3.4.7 Variational Autoencoder....70
3.4.8 GAN....70
3.4.9 Network Architecture and Training Configuration....71
3.4.10 Independent System Verification....71
3.4.11 Data Leakage Controls....72
3.5 Phase 4: Model Training and Testing....72
3.5.1 Train-Test Split....72
3.5.2 Training Data and Class Imbalance....72
3.5.3 Model Fitting....73
3.5.4 Testing and Anomaly Scoring....73
3.5.5 Independent System Verification....73
3.5.6 Data Leakage and Reproducibility Controls....73
3.6 Phase 5: Evaluation Metrics....74
3.6.1 Reporting and Interpreting Evaluation Results....74
3.6.2 Confusion-Matrix Interpretation....74
3.7 Extended Comparative Evaluation Criteria....75
3.7.1 Performance Consistency....75
3.7.2 Automation....75
3.7.3 Training Time....75
3.7.4 Interpretability and Scalability....75
3.8 Reproducibility and Integrated Experimental Workflow....76
Chapter 4: Results and Discussion....77

4.1 Baseline Performance (HDFS v1) 77
4.2 Traditional Machine Learning Results: 77
4.3 Deep Learning (Supervised) Results: 77
4.4 Generative AI Results: 78
Comparative Analysis Synthesis (HDFS): 78
4.5 Spark Logs Analysis 79
4.6 Spark Comparative Analysis Synthesis: 80
4.7 Hadoop Logs Analysis 82
4.8 Hadoop Comparative Analysis Synthesis: 82
4.9 Overall Performance Synthesis (Spark vs. Hadoop) 83
4.10 Generative Model Failures 84
4.10.1 GANs: Structural Mismatch 84
4.10.2 VAEs: Blurry Reconstruction 84
4.11 Key Takeaways from the Data: 84
Chapter 5: Conclusions and Recommendations 86
5.1 Thesis Core Findings and Contributions 86
5.2 The Performance Trade-Off and Research Outcomes 86
5.3 System-Specific Degradation and Operational Boundaries 87
5.4 Recommendations for Future Work 87

List of Figures

Figure 1 Methodology 64
Figure 3 Spark logs Model Comparison 81
Figure 3 Performance Matrix Spark vs Hadoop 84

List of Tables

Table 1 Dataset comparison 47
Table 2 Comparative Summary of Logs 52
Table 3 Literature Review and Preliminary Benchmarking 54

Table 4 HDFS All Models Comparison.....79
Table 7 Spark Logs All Model Results Summary 81
Table 8 Spark vs Hadoop All Models Results.....83

Symbols and Abbreviations

Abbreviation	Definition
AI	Artificial Intelligence
ANN	Artificial Neural Network
AUC	Area Under the Curve
DL	Deep Learning
ELBO	Evidence Lower Bound
FN	False Negative
FP	False Positive
GAN	Generative Adversarial Network
GRU	Gated Recurrent Unit
HDFS	Hadoop Distributed File System
KL	Kullback-Leibler
LSTM	Long Short-Term Memory
ML	Machine Learning
NDM	Next Day Monitoring
NLP	Natural Language Processing
OOD	Out-of-Distribution
PCA	Principal Component Analysis
RNN	Recurrent Neural Network
ROC	Receiver Operating Characteristic
TN	True Negative
TP	True Positive
VAE	Variational Autoencoder
YARN	Yet Another Resource Negotiator

Symbol	Description
E	Parsed log sequence
et	Feature embedding of a log event at time step t
ht	Hidden state of the recurrent network at time step t
T	Total length of the temporal log sequence
V	Vocabulary size of possible unique log events
W,b	Weight matrix and bias vector of a neural network layer
z	Latent space vector representation
x	Input data (e.g., original log sequence)
y	True label (used in supervised benchmarking)
y^	Predicted output or reconstructed sequence
Rd	d-dimensional real coordinate space (used for embeddings)
E	Expected value / Expectation
pθ(x|z)	Probabilistic decoder parameterized by neural network weights θ
qϕ(z|x)	Probabilistic encoder parameterized by neural network weights ϕ
μ	Mean vector of the learned latent distribution
Σ	Covariance matrix of the learned latent distribution
N(μ,Σ)	Multivariate Normal (Gaussian) distribution
LVAE	Evidence Lower Bound (ELBO) loss function for VAE
DKL	Kullback-Leibler divergence
A(x)	Anomaly score based on reconstruction loss
τ	Dynamic anomaly detection threshold
‖·‖2	Squared L2 norm (Euclidean distance, used in reconstruction error)
∇	Gradient operator (used in optimization and backpropagation)
α	Learning rate during model training

Chapter 1: Introduction

1.1 System Logs

Modern computing environments continuously generate large amounts of information describing system operations, user activities, application behavior, network communication, and security-related events. One of the most important sources of this information is system logs. According to the National Institute of Standards and Technology (NIST), a log is a record of events occurring within an organization's computing assets, including physical and virtual systems, networks, services, and cloud environments [1]. Log entries contain information associated with specific events that occur within a system or network.

System logs are generated by a wide range of sources, including operating systems, servers, applications, routers, switches, firewalls, authentication systems, intrusion detection systems, and other security and network devices [1]. These logs provide information about the operational state of a system and can be used for monitoring, troubleshooting, auditing, security analysis, and incident investigation.

A typical log entry may contain several attributes, including a timestamp, event type, severity level, username, IP address, process identifier, service name, and descriptive message. The exact structure depends on the logging mechanism and the system generating the event.

For example:

2026-08-30 20:15:42 SSH Failed password user=asad source=192.168.1.50

This log entry provides information about the time of the event, the type of activity, the user account involved, and the source address. Consequently, system logs provide a historical record that can help administrators understand what happened within a computing environment.

Previous research has also established that system logs are an important source of information for understanding system status, diagnosing failures, and detecting anomalous behavior [2]. Du et al. described system logs as valuable sources for monitoring system status and identifying performance problems and failures.

Why Are System Logs Generated?

System logs are generated to maintain a record of important events and activities occurring within computing systems. Effective log management enables organizations to generate, transmit, store, access, analyze, and retain log information for operational and security purposes [1]. NIST identifies log management as an important component of cybersecurity and system monitoring.

The major purposes of generating system logs include the following.

1.1.1 Troubleshooting

System logs provide information that can help administrators identify errors, failures, and unexpected system behavior. When an application crashes or a service becomes unavailable, log records can help determine the sequence of events that occurred before the failure.

Log analysis can therefore reduce the time required to identify the root cause of operational problems.

1.1.2 Security Monitoring

Logs provide important evidence for detecting unauthorized access and potentially malicious activities. Authentication failures, privilege changes, suspicious network connections, and other security-related events can be recorded and subsequently analyzed.

NIST specifically identifies logging as an important mechanism for detecting and investigating cybersecurity incidents [1].

1.1.3 System and Network Monitoring

Logs can be used to monitor the operational condition of servers, applications, network devices, and other computing resources. They can provide information about service availability, system errors, network connections, and configuration changes.

1.1.4 Auditing

Logs provide a historical record of activities performed within a system. This can help organizations determine what happened, when it happened, and which account or system was involved.

1.1.5 Incident Investigation

After a security incident or system failure, historical logs can be analyzed to reconstruct the sequence of events. This information can support root-cause analysis and forensic

investigation [2]. DeepLog also highlights the importance of system logs for debugging failures and performing root-cause analysis.

1.2 Types of System Logs

System logs can be categorized according to the component that generates them and the type of information they contain.

1.2.1 Authentication Logs

Authentication logs record activities associated with user authentication. These may include successful login attempts, failed login attempts, password changes, account lockouts, and other authentication-related events.

For example:

Failed password for admin from 192.168.1.50

Repeated authentication failures within a short period may represent abnormal behavior and can potentially indicate a brute-force attack.

1.2.2 Application Logs

Application logs are generated by software applications to record their execution and operational behavior. They may include application startup and shutdown events, exceptions, errors, warnings, transactions, and other application-specific activities.

1.2.3 Operating-System Logs

Operating-system logs record events generated by the underlying operating system. These can include system startup and shutdown events, process failures, service status changes, hardware-related errors, and system warnings.

1.2.4 Security Logs

Security logs contain information associated with security-related activities, including unauthorized access attempts, permission changes, authentication events, firewall activities, and other potentially malicious behavior.

1.2.5 Network Logs

Network devices such as routers, switches, firewalls, VPN gateways, and intrusion detection systems generate network-related logs. These records can contain information about connections, blocked traffic, routing events, DNS requests, VPN sessions, and firewall activities.

NIST identifies operating-system logs, application logs, and cybersecurity software logs among the important sources of organizational log data [1].

1.3 Importance of Log Analysis

Although system logs contain valuable information, modern systems can generate extremely large quantities of log data. The volume and diversity of log messages make manual analysis difficult and time-consuming [3]. Research on log analysis has therefore increasingly focused on automated techniques for extracting useful information from log data.

The problem is particularly challenging because logs are often unstructured or semi-structured and may originate from heterogeneous systems. Different applications and devices can use different formats, message structures, and terminology [3].

As a result, automatic log analysis is increasingly necessary for identifying important events, system failures, and abnormal behaviour.

1.4 Anomalies in System Logs

An anomaly is an observation or behavior that deviates significantly from an expected or normal pattern.

In system-log analysis, an anomaly may appear as an unusual log event, an abnormal frequency of events, or an unexpected sequence of log messages. The identification of such behavior is important because anomalous events may indicate system failures, software problems, configuration errors, or security incidents.

DeepLog demonstrated that normal log-event sequences can be learned from historical system execution and that deviations from these learned patterns can be used to identify anomalies [2].

For example, consider:

10:01:02 Successful login

10:01:15 Successful login

10:01:20 Failed login

10:01:21 Failed login

10:01:22 Failed login

10:01:23 Failed login

10:01:24 Failed login

While a single failed login may represent normal user behavior, a sequence of repeated failures occurring within a very short period may indicate abnormal activity.

1.5 Types of Log Anomalies

Log anomalies can generally be classified into several categories.

Point Anomalies

A point anomaly occurs when an individual log event significantly differs from normal observations.

For example, an unusual system event that rarely occurs during normal operation may be considered a point anomaly.

1.5.1 Contextual Anomalies

A contextual anomaly occurs when an event is abnormal under a particular context, even though the same event may be normal under different circumstances.

For example, a login at an unusual time may be considered anomalous depending on the normal behavior of the user.

1.5.2 Collective Anomalies

A collective anomaly occurs when a sequence or group of individually normal events forms an abnormal pattern.

This category is particularly important for system logs because many system problems cannot be detected by examining individual events alone. Instead, the relationship between consecutive or related events must be considered.

Research such as DeepLog and LogAnomaly specifically addresses sequential patterns in log data [2], [4].

1.6 Challenges in Log Anomaly Detection

The detection of anomalies in system logs presents several challenges.

1.6.1 Large Volume of Data

Modern computing environments generate large amounts of log information. Manual inspection of such data is impractical, creating a need for automated analysis techniques [3].

1.6.2 Diverse Log Formats

Logs can originate from different operating systems, applications, network devices, and security tools. Consequently, their formats and structures can vary significantly.

Unstructured and Semi-Structured Data

Many log messages contain both structured fields and natural-language descriptions. Extracting meaningful features from such data is therefore challenging.

1.6.3 Sequential Dependencies

The meaning of an event can depend on the events that occurred before it. For example, a successful login following many failed attempts may be more suspicious than an isolated successful login.

1.6.4 Changing System Behavior

Normal system behavior may change because of software updates, workload changes, configuration modifications, and changes in user behavior.

1.6.5 Unknown Anomalies

A particularly difficult problem is the detection of anomalies that have not previously been observed. A model trained only on known anomaly patterns may not recognize new types of abnormal behavior.

A recent survey of deep-learning approaches emphasizes the challenges associated with log diversity, preprocessing, feature representation, and selecting appropriate neural architectures for different log-analysis tasks [3].

1.7 Artificial Intelligence for Log Anomaly Detection

The limitations of manual and rule-based log analysis have motivated the use of Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL) techniques.

Traditional approaches may rely on manually defined rules, thresholds, or predefined patterns. Although these techniques can be effective for known conditions, they may have difficulty identifying complex or previously unseen patterns.

Deep-learning approaches can instead learn representations of normal system behavior from historical log data. For example, DeepLog uses a Long Short-Term Memory (LSTM) neural network to model log events as sequences and detect deviations from learned patterns [2].

Similarly, LogAnomaly uses semantic representations of log templates and considers both sequential and quantitative anomalies [4]. The authors demonstrated that the approach can identify different forms of anomalous behavior while reducing some false alarms associated with newly appearing log templates.

More recently, Transformer-based approaches have also been investigated. LogBERT, for example, applies BERT-based self-supervised learning to model normal log sequences and identify deviations from those patterns [5].

A systematic survey of deep-learning methods for log anomaly detection shows that researchers have investigated architectures including RNNs, CNNs, autoencoders, GANs, Transformers, attention mechanisms, and graph neural networks [3].

1.8 Motivation of the Study

The increasing complexity of modern computing environments has resulted in a continuous increase in the amount and diversity of system-generated log data. Although these logs contain valuable information about system operation, security, and failures, their large volume makes manual analysis impractical [1], [3].

Furthermore, existing anomaly detection techniques face challenges associated with changing log structures, sequential dependencies, noisy data, and previously unseen anomalous behavior.

These challenges have motivated researchers to investigate intelligent approaches capable of automatically learning patterns from system logs. Deep-learning approaches such as DeepLog and LogAnomaly demonstrated the potential of learning normal log behavior automatically [2], [4]. Transformer-based approaches such as LogBERT have further explored contextual representations for log anomaly detection [5].

Therefore, this research investigates intelligent approaches for system-log anomaly detection, with particular emphasis on identifying abnormal patterns from large-scale log data.

1.9 Importance of Log Anomaly Detection

System logs provide valuable information about the operational and security status of computing systems. However, modern systems continuously generate large volumes of logs from different sources, making manual inspection difficult and time-consuming. Therefore, automated log anomaly detection is important for identifying abnormal behavior, system failures, and potential security threats in a timely manner.

Log anomaly detection identifies log events or sequences that deviate from expected system behavior. Such deviations may indicate security attacks, software failures, configuration problems, resource issues, or other abnormal conditions. Automated detection can help organizations identify these problems earlier and allow administrators to focus their attention on the most relevant events.

1.9.1 Security and Early Warning

One of the most important applications of log anomaly detection is **cybersecurity**. Authentication logs, access-control logs, firewall logs, network logs, and application logs can contain evidence of unauthorized or suspicious activities.

For example, a system may normally experience a small number of failed login attempts each day. If hundreds of failed authentication attempts occur within a few minutes, the behavior may indicate a brute-force attack or another unauthorized access attempt.

Early detection of suspicious activities can allow security teams to investigate and respond before an incident causes significant damage. NIST's incident-response guidance emphasizes the importance of detecting and analyzing incidents and responding to them effectively [8].

Log anomaly detection can therefore assist in identifying:

- Repeated failed authentication attempts
- Unauthorized access attempts
- Suspicious user behavior
- Unusual network connections
- Abnormal application activity
- Unexpected privilege changes
- Potential malware-related activity

1.9.2 Detection of System Failures

Log anomaly detection is also important for identifying operational and system failures. Applications, operating systems, and network devices continuously generate logs describing their operational status.

Abnormal patterns in these logs may indicate:

- Application crashes
- Service failures
- Hardware problems
- Network failures
- Resource exhaustion
- Configuration errors
- Unexpected application behavior

Meng et al. emphasized that detecting anomalies in system logs is important for the timely identification of system malfunctions. They also highlighted that manually identifying anomalies in large volumes of logs is time-consuming, error-prone, and infeasible [9].

Consequently, automated anomaly detection can help administrators identify operational problems without manually examining every log entry.

1.9.3 Reduction of Manual Log Monitoring

Modern computing infrastructures can generate extremely large quantities of log data. Logs may be produced continuously by servers, operating systems, applications, databases, network devices, firewalls, and security systems.

Manually examining all these records is impractical. Automated anomaly detection can act as a filtering mechanism by identifying potentially abnormal events and reducing the amount of information that human analysts need to examine.

For example, instead of manually reviewing thousands of authentication records, an anomaly detection system can identify an unusual pattern such as:

Normal: 10 failed login attempts/day

Observed: 500 failed login attempts in 5 minutes

Result: Potential anomaly → Alert generated

This allows system administrators and security analysts to concentrate on events that require further investigation.

1.9.4 Faster Incident Response

Another important advantage of anomaly detection is its ability to support **faster incident response**.

The general process can be represented as:

Log Generation → Anomaly Detection → Alert → Investigation → Response

When an abnormal pattern is automatically detected, an alert can be generated for further investigation. This reduces the time between the occurrence of abnormal behavior and the beginning of the investigation.

NIST's incident-handling guidance identifies detection, analysis, and appropriate response as important components of effective incident handling [8].

Early detection can therefore help organizations minimize the potential impact of security incidents and system failures.

1.9.5 Support for Incident Investigation and Auditing

Log anomaly detection also supports incident investigation and auditing. Once an anomaly is detected, the underlying log records can be examined to reconstruct the sequence of events.

Investigators may determine:

1. When the abnormal activity started.

2. Which user or system was involved.
3. Where the activity originated.
4. What events occurred before the anomaly.
5. Which systems or services were affected.
6. What activities occurred after the initial event.

Historical log information can therefore provide evidence for understanding the cause and impact of an incident.

NIST guidance emphasizes the importance of analyzing incident-related information to determine appropriate responses and support effective incident handling [8].

1.9.6 Detection of Sequential and Collective Anomalies

An important characteristic of system-log anomalies is that abnormal behaviour is not always represented by a single log entry. Sometimes, a sequence of individually normal events can collectively represent anomalous behaviour.

For example:

Login failure

Login failure

Login failure

Login failure

Login failure

Successful login

Each individual failed login may be legitimate. However, the complete sequence may indicate a potential brute-force attack followed by successful account access.

Research has therefore investigated methods that consider the sequence and frequency of log events rather than analyzing individual events independently.

LogAnomaly, for example, was developed to detect both sequential and quantitative anomalies in unstructured logs [9].

This demonstrates the importance of considering relationships between log events when designing automated anomaly detection systems.

Importance for Intelligent Log Analysis

The limitations of manual log monitoring have motivated the development of machine-learning and deep-learning approaches for automated log analysis.

Deep-learning approaches can learn patterns of normal system behavior from historical log data and identify deviations from those patterns. DeepLog, for example, models

system log sequences using deep learning to detect anomalous behavior and diagnose system problems [10].

Similarly, LogAnomaly models log streams as sequences and uses semantic information from log templates to identify sequential and quantitative anomalies [9].

More recent approaches have investigated Transformer-based models. NeuralLog, for example, addresses limitations associated with traditional log parsing by extracting semantic representations directly from raw log messages and using a Transformer-based model to capture contextual information from log sequences [11].

These developments demonstrate the increasing importance of intelligent approaches for automatically analyzing large-scale and complex log data.

1.9.7 Overall Importance

The importance of log anomaly detection can therefore be summarized into several major areas:

Area	Importance
Security	Detects suspicious and unauthorized activities
Early Warning	Identifies abnormal behavior before major damage occurs
System Reliability	Helps identify failures and operational problems
Automation	Reduces the need for manual log inspection
Incident Response	Supports faster detection and investigation
Auditing	Provides historical evidence of system activities
Scalability	Enables analysis of large volumes of log data
Intelligent Monitoring	Uses ML/DL to learn complex patterns

Thus, log anomaly detection plays an important role in maintaining the security, reliability, availability, and operational efficiency of modern computing environments.

1.9.8 System Logs in Distributed Architectures: Hadoop, HDFS, and Apache Spark

1.10 Introduction to Distributed System Observability

In the contemporary landscape of big data and large-scale computational frameworks, system observability is fundamentally driven by system logs. Logs are automatically generated, time-stamped textual records that continuously capture operational events, state transitions, and execution anomalies across host environments. Operating entirely asynchronously, logging frameworks provide an immutable audit trail—a

chronological “diary” of the system—without introducing the computational overhead or execution interruptions typically associated with active runtime profiling.

As computational architectures have evolved from monolithic servers to distributed clusters, the volume, velocity, and variety of system logs have expanded exponentially. Frameworks such as Apache Hadoop and Apache Spark orchestrate tasks across hundreds or thousands of concurrent nodes. Consequently, relying on manual inspection or traditional rule-based heuristics to identify systemic failures, resource exhaustion, or security breaches is no longer viable. This paradigm shift relies on structuring raw log data into sequential templates, which subsequently serve as the foundational training data for artificial intelligence pipelines designed to autonomously map baseline system behavior and detect runtime anomalies.

1.11 The Hadoop Ecosystem and Resource Management Logs

Apache Hadoop is the foundational open-source framework designed to store and process massive datasets across distributed clusters of commodity hardware. At the core of its operational management is YARN (Yet Another Resource Negotiator), which acts as the cluster’s distributed operating system.

Hadoop YARN logs are dedicated to documenting the negotiation, allocation, and monitoring of cluster computing resources across competing applications. These logs capture highly specific telemetry regarding the infrastructure layer. Key events recorded include ApplicationMaster registrations, container launch and teardown sequences, task preemptions, and NodeManager health statuses. By parsing these logs, it is possible to extract critical variables such as Application IDs, container execution times, and CPU/memory resource requests.

Analyzing Hadoop resource logs is essential for identifying infrastructure-level anomalies. For instance, scheduling deadlocks, failed container initializations, or cluster-wide resource starvation are explicitly detailed within these log sequences. Identifying these patterns is a necessary precursor to maintaining high availability in enterprise data processing environments.

1.12 HDFS Logs: The Storage Layer

The Hadoop Distributed File System (HDFS) serves as the primary storage layer of the Hadoop ecosystem. It operates by splitting massive datasets into smaller, manageable blocks and distributing them redundantly across multiple physical server nodes (DataNodes), managed by a central directory node (NameNode). This redundancy

ensures fault tolerance; if a single hardware node fails, the data is preserved elsewhere in the cluster.

HDFS logs continuously record the lifecycle of these physical data blocks. They capture NameNode directives and DataNode operations, including block allocations, data pipeline creations, chunk replication successes or failures, and regular node heartbeat synchronizations. A standard HDFS log entry might read: "Receiving block blk_-1608999687919862906 src: /10.251.106.10:35023 dest: /10.251.106.10:50010". These logs are heavily structured around specific identifiers, primarily the block ID, source IP address, and destination IP address.

Because HDFS operations—such as allocating, replicating, and verifying a data block—strictly follow a chronological lifecycle, the resulting logs form highly predictable sequences during normal operation. When an anomaly occurs, such as an allocated block failing to replicate across the required number of DataNodes due to a network partition, the sequence of expected log events is broken. This inherently sequential nature makes HDFS logs an ideal dataset for evaluating time-series and sequence-prediction algorithms.

1.13 Apache Spark Logs: The Computational Layer

While HDFS manages the persistence of data, Apache Spark handles the high-speed, in- memory processing of that data. Spark frequently runs on top of HDFS, ingesting the stored blocks and performing complex transformations and analytics in RAM. This in-memory execution model makes Spark significantly faster than traditional disk-bound map-reduce frameworks.

Apache Spark logs detail the computational execution of these in-memory jobs, focusing on task scheduling and dynamic resource utilization. The logs capture the construction of Directed Acyclic Graphs (DAGs), the evaluation of Resilient Distributed Dataset (RDD) lineages, stage boundary calculations, task serialization, and executor memory states. Key variables extracted during parsing include Executor IDs, memory allocation metrics (such as bytes spilled to disk), partition numbers, and Garbage Collection (GC) execution times. Unlike HDFS logs, which track relatively linear storage operations, Spark logs exhibit high variance and complexity in their execution paths. Deviations in these log sequences typically indicate computational bottlenecks, severe data skew during dataset transformations, or fatal Out-Of-Memory

(OOM) errors. The highly dynamic nature of Spark logs presents a unique challenge for automated anomaly detection, as normal baseline behavior encompasses a vast array of valid execution paths.

1.14 Rationale for Inclusion in Anomaly Detection Research

The selection of HDFS, Apache Spark, and Hadoop log datasets is critical for establishing a robust, empirical evaluation framework for system log anomaly detection. These datasets provide the massive scale, structural complexity, and real-world noise required to rigorously benchmark a wide spectrum of algorithmic approaches.

Traditional machine learning models offer essential baseline performance metrics for static log template analysis. Algorithms such as Support Vector Machines (SVM), Decision Trees, and XGBoost are highly effective at classifying logs based on extracted feature vectors, such as event frequencies or term occurrences (TF-IDF). Furthermore, Isolation Forests are particularly suited for detecting metric-based anomalies within these

logs, such as unexpected memory spikes recorded in Spark executor telemetry. However, these traditional models frequently fall short in capturing the temporal dependencies between events.

The inherent sequential nature of HDFS block replication and Spark DAG execution necessitates the application of advanced deep learning architectures. Recurrent neural networks are designed specifically for sequential data. By treating log entries as elements governed by specific execution "grammar rules," Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRU) can map the temporal dependencies of normal execution paths. Moreover, Bi-directional LSTMs (Bi-LSTM) analyze log sequences from both past-to-future and future-to-past contexts, providing a holistic understanding of the execution state and significantly reducing false positives when identifying complex state transitions.

Finally, production log datasets from Hadoop and Spark exhibit severe class imbalance; enterprise systems operate normally the vast majority of the time, resulting in a scarcity of anomalous training data. This imbalance causes standard classifiers to overfit to the

normal baseline. To address this fundamental challenge, generative artificial intelligence architectures must be evaluated. Generative Adversarial Networks (GAN) can be deployed to synthesize minority-class log sequences, effectively balancing the dataset and improving boundary definitions for downstream classifiers. Concurrently, Variational Autoencoders (VAE) can compress healthy log distributions into a latent space; anomalies are subsequently flagged based on high reconstruction errors when processing anomalous HDFS or Spark sequences.

In conclusion, evaluating this comprehensive spectrum of models—from traditional XGBoost and Isolation Forests to sequence-based LSTMs and GRUs, and finally to generative GANs and VAEs—against the specific telemetry of Hadoop, HDFS, and Spark logs provides a definitive assessment of modern AI-driven system observability.

1.15 Problem Statement

As distributed computing environments, such as Hadoop and Apache Spark, continue to scale in complexity, system logs remain the primary diagnostic artifact for troubleshooting and performance monitoring. However, the sheer volume, velocity, and unstructured nature of these logs have rendered manual, rule-based inspection obsolete. Consequently, current industry standards for automated log anomaly detection are paralyzed by a critical functional dichotomy between accuracy and adaptability.

- **Brittleness of Supervised ML:**

Traditional discriminative models, including Support Vector Machines (SVM) and extreme gradient boosting (XGBoost), can achieve exceptionally high detection accuracy in closed-world scenarios. However, they are fundamentally brittle. These architectures are entirely dependent on rigid, manual feature engineering (e.g., Event Count Matrices) and extensive, pre-labeled anomaly datasets. This reliance causes them to fail catastrophically in the face of concept drift—such as when log syntax naturally evolves during software updates, or when models are ported across heterogeneous system architectures.

- **Inefficacy of Unsupervised Baselines:**

Conversely, traditional unsupervised techniques (e.g., Isolation Forest, One-Class SVM) attempt to bypass the need for labeled data by isolating statistical outliers. Yet, they consistently struggle to map the complex, temporal, and sequential dependencies inherent in distributed execution traces. This structural blindness results in unacceptably high false-positive rates and poor detection efficacy when encountering

novel, unknown error patterns (zero-day anomalies) within noisy, highly concurrent environments. There is a critical lack of a unified framework that combines the high accuracy of supervised learning with the flexibility of unsupervised discovery.

Ultimately, there remains a critical lack of a robust, unified framework capable of bridging this operational gap. Existing paradigms force a compromise: engineering teams must choose between the high precision of supervised learning—which demands unsustainable manual labeling and feature extraction—and the flexibility of unsupervised discovery, which currently lacks the contextual depth to operate reliably across diverse, real-world systems without triggering alert fatigue.

B. Objectives and Research Questions

This research aims to benchmark traditional supervised and unsupervised models on the HDFS, Hadoop and Spark datasets to establish a baseline for known error detection. It evaluates discriminative Deep Learning (DL) sequence models for anomaly detection without manual feature extraction, and validates Generative AI architectures to determine their superiority in detecting unknown anomalies and their performance consistency across distinct platform datasets. Ultimately, it conducts a tri-fold comparative study producing a definitive performance matrix comparing ML, DL, and Generative AI.

The core research questions are:

- **RQ1:** To what extent can Generative AI architectures maintain consistent anomaly detection performance across distinct, heterogeneous distributed systems (e.g., evaluated independently on HDFS and Spark) compared to Traditional Supervised models?
- **RQ2:** Can Generative Deep Learning models eliminate the dependency on manual feature engineering required by traditional AI, while maintaining high F1-scores?
- **RQ3:** Can deep generative models significantly outperform traditional unsupervised baselines in detecting unknown 'zero-day' anomaly patterns?

1.16 Motivation

The rapid growth of distributed systems like Hadoop and Spark has made manual log monitoring impossible. Traditional machine learning requires tedious manual labeling and struggles to detect unknown, zero-day errors. Therefore, this research is motivated

by the critical need to automate anomaly detection using Generative AI, enabling systems to identify novel failures without relying on labeled error data or manual feature extraction.

1.17 Objectives

My objectives for carrying out this research were:

a. **Comprehensive Paradigm Comparison:** Evaluate and compare the performance of Traditional Machine Learning, sequence-based Deep Learning (e.g., LSTMs), and Generative AI models (e.g., VAEs, GANs) for automated log anomaly detection.
b. **Elimination of Manual Engineering:** Assess the capability of deep sequence architectures to remove the reliance on labor-intensive manual feature extraction (such as Event Count Matrices) and heavily labeled datasets.
c. **Zero-Day Anomaly Detection:** Validate the effectiveness of Generative AI models in discovering unknown, novel errors by operating under a strictly normal-only training paradigm.
d. **Isolated Operational Benchmarking:** Establish accurate performance baselines by independently training and testing models within distinct system boundaries (HDFS, Apache Spark, and OpenStack) to prevent cross-system data contamination.
e. **Trade-off and Boundary Analysis:** Delineate the operational limits of these architectures, specifically analyzing the trade-offs between detection precision, automation levels, and noise resilience in highly concurrent environments like Spark.

1.18 Scope of the thesis

What This Thesis Covers

- **Architectural Evaluation:** A direct comparative analysis of three specific paradigms: Traditional Machine Learning (supervised and unsupervised), sequence-based Deep Learning (e.g., LSTMs), and Generative AI models (e.g., VAEs, GANs).
- **Specific Distributed Systems:** The application of these models to massive, unstructured log collections originating exclusively from Hadoop (HDFS), Apache Spark, and OpenStack environments.

- **Zero-Day Detection & Normal-Only Training:** An investigation into how Generative AI can identify novel, unknown anomalies by training strictly on "normal" system behavior, bypassing the need for labeled error datasets.
- **Isolated Benchmarking:** System-specific training and testing. Models are evaluated within the boundaries of their respective datasets (e.g., Spark models tested only on Spark data) to establish true baselines without cross-system contamination.
- **Performance Trade-offs:** An analysis of the operational boundaries of these models, specifically focusing on the balance between detection precision, automation capability, and resilience to noise in concurrent systems.

What This Thesis Does Not Cover

- **Automated Remediation:** The scope is strictly limited to the *detection* of anomalies. This research does not cover automated root-cause resolution, self-healing mechanisms, or system recovery protocols.
- **Non-Log System Metrics:** The study focuses exclusively on unstructured textual log data. It does not analyze other telemetry data such as hardware metrics (CPU, RAM, disk I/O) or network packet traffic.
- **Real-Time Production Latency:** While operational boundaries are discussed, optimizing these models for millisecond-level, real-time streaming deployment in live production environments is outside the scope of this study.
- **Other Cloud Platforms:** The evaluation is restricted to HDFS, Spark, and OpenStack. Other distributed orchestrators or commercial cloud environments (e.g., Kubernetes, AWS, Azure) are not included in the dataset analysis.

1.19 Thesis Layout

The thesis is arranged in this way. In Chapter 1, an Introduction is presented outlining the motivation, objectives, and scope of the research. The remainder of the thesis is structured as follows:

- **Chapter 2: Literature Review.** Presents a comprehensive survey of prior research in automated log parsing, traditional machine learning techniques for system monitoring, and the recent shift toward deep learning and Generative AI for anomaly detection.
- **Chapter 3: Methodology and Architecture.** Details the core technical approach, including the preprocessing of unstructured log data, and describes

the architectural design of the evaluated models—spanning traditional algorithms, sequence-based Deep Learning (e.g., LSTMs), and Generative AI frameworks (e.g., VAEs, GANs).

- **Chapter 4: Experimental Setup.** Outlines the specific datasets utilized (HDFS, Apache Spark, and OpenStack) and defines the isolated training and testing protocols established to ensure true operational baselines without cross-system contamination.
- **Chapter 5: Results and Discussion.** Provides a rigorous comparative analysis of model performance. This chapter evaluates the efficacy of zero-day anomaly discovery via normal-only training and examines the trade-offs between precision and noise resilience in highly concurrent environments.
- **Chapter 6: Conclusion and Future Work.** Summarizes the key findings, highlights the operational boundaries of the evaluated paradigms, and proposes potential avenues for future research in automated system reliability.

Chapter 2: Literature Survey

2.1 Introduction to the Literature Review

System logs are a fundamental source of operational information in modern computing environments. They record events produced by operating systems, applications, middleware, databases, distributed frameworks, network services, and security components. As software infrastructure becomes larger and more dynamic, the volume and diversity of log records increase rapidly. Manual inspection is therefore difficult, especially when an operator must identify a small number of abnormal events among millions of routine messages. Automated log anomaly detection has consequently become an important research problem at the intersection of machine learning, deep learning, natural language processing, and distributed systems [12], [17].

The literature supplied for this thesis describes a clear evolution from traditional machine-learning techniques to deep sequence models and, more recently, generative and Transformer-based architectures. Early methods generally converted unstructured log messages into structured event representations and then applied statistical or supervised learning algorithms. Later approaches learned temporal relationships between event sequences with recurrent neural networks. The newest approaches increasingly attempt to capture semantic context, previously unseen events, and normal-system distributions with self-supervised or generative learning [17], [21], [22].

This chapter expands the existing literature survey by discussing the principles, strengths, limitations, datasets, evaluation criteria, and research gaps associated with these three generations of approaches. Particular attention is given to the HDFS, BGL, Hadoop, Apache Spark, and OpenStack environments because they represent different operational characteristics. HDFS contains structured distributed-file-system activity, BGL represents large-scale supercomputer logs, while Spark and OpenStack introduce highly concurrent and heterogeneous execution behavior. These differences are important because a model that performs well on one dataset may not necessarily retain the same behavior on another.

The central theme of the review is not simply which model produces the highest F1-score on a benchmark. Instead, the literature is examined in terms of data requirements, feature engineering, generalization, robustness to changing log formats, ability to operate without anomaly labels, computational complexity, and suitability for zero-day

anomaly detection. This perspective supports the thesis objective of comparing Traditional ML, Deep Learning, and Generative AI under controlled dataset-specific conditions.

2.2 System Logs and Their Role in Anomaly Detection

A system log is a chronological record of events generated while a computing system performs its operations. A typical record may contain a timestamp, severity level, component identifier, process identifier, message template, and variable parameters. For example, a message such as "Block <id> received from <node>" contains a relatively stable event template and changing parameter values. Log records can therefore be considered semi-structured data: they are textual, but repeated operational events often follow recognizable templates [17].

Logs are valuable for anomaly detection because they provide an indirect representation of system behavior. A healthy system tends to execute recurring workflows, and these workflows generate recurring sequences of log events. A failure, performance degradation, configuration error, or unexpected interaction can alter this sequence. The anomaly may appear as an unusual event, an unexpected event order, an abnormal parameter value, an unusual frequency, or a combination of individually normal events occurring in an abnormal context.

The usefulness of logs also introduces a major challenge. Unlike conventional tabular datasets, logs are rarely produced in a uniform analytical format. Different applications use different message structures, terminology, timestamp formats, and severity conventions. Software updates can introduce new messages or modify existing templates. Distributed systems further interleave messages from many components, making temporal relationships difficult to interpret. Log anomaly detection must therefore transform noisy, heterogeneous textual streams into representations that preserve the information required for reliable detection [18].

The literature demonstrates that preprocessing is not a minor implementation detail. Parsing quality can directly affect downstream anomaly detection accuracy. Recent empirical work has shown that different parsing algorithms can lead to different detection performance even when the anomaly detector itself remains unchanged [18]. This observation is particularly important for comparisons between traditional and deep models because improvements attributed to the detector may partly originate from differences in preprocessing.

2.3 Types of Log Anomalies

Log anomalies can be classified according to the way in which normal system behavior is violated. A point anomaly is an individual event that is unusual relative to normal observations. A contextual anomaly is abnormal only under a particular context, such as a message occurring at an inappropriate time or within an unexpected component. A collective or sequential anomaly is produced by a group of events whose combined behavior is abnormal even though each event may appear normal individually.

Sequential anomalies are particularly important in distributed systems. Suppose a normal execution follows event A, then B, then C. If event A is followed by C without B, the individual events may all be valid, but their order violates the learned execution pattern. DeepLog formulated anomaly detection as a sequence prediction problem, where an event is considered suspicious when the observed next event is inconsistent with the model's learned normal sequence [20]. This formulation became influential because it directly models temporal dependencies instead of relying only on event frequencies.

Quantitative anomalies provide another category. A system may generate a normal event type but at an abnormal frequency. For example, repeated retry, authentication, connection, or block-replication messages may indicate a failure even when the message template itself is legitimate. LogAnomaly explicitly considered both sequential and quantitative anomaly characteristics, demonstrating the importance of combining event ordering with frequency-related information [21].

Parameter anomalies are also important. A log template can remain unchanged while one of its variable values becomes abnormal. HitAnomaly addressed this issue by modeling both log-template sequences and parameter values using a hierarchical Transformer structure [23]. This direction shows why a detector that uses only event IDs can miss important information embedded in the variable fields of log messages.

2.4 Traditional Rule-Based Monitoring

Before machine learning became common, system monitoring frequently depended on manually defined rules. Operators searched for keywords such as ERROR, FAIL, WARNING, TIMEOUT, or EXCEPTION, or created threshold rules for counters and resource measurements. Such methods are easy to interpret and can be effective when

failure signatures are stable and known. However, they require experts to anticipate the conditions that should trigger an alert.

Rule-based monitoring is inherently reactive. If a new failure generates a message that does not match an existing rule, the system may not raise an alert. Similarly, a keyword may occur during normal operation and create a false alarm. In distributed systems, the meaning of a message often depends on surrounding events, so a single keyword is rarely sufficient to characterize system state. These limitations motivated the transition toward data-driven anomaly detection [17].

Another limitation is maintenance cost. As applications evolve, log messages change, new services are introduced, and existing workflows become more complex. Each change can require new rules or modifications to existing ones. The problem becomes especially difficult in large-scale cloud environments where many services are updated independently. Automated learning methods attempt to reduce this burden by learning regularities from historical operational data rather than requiring every normal and abnormal condition to be manually specified.

Nevertheless, rule-based monitoring should not be considered obsolete. In practical operations, deterministic rules remain useful for high-confidence conditions, compliance checks, known security events, and safety-critical alerts. A mature monitoring architecture may therefore combine rules with machine-learning detectors, using the former for deterministic signatures and the latter for previously unknown or context-dependent anomalies.

2.5 Traditional Machine Learning: PCA and Statistical Methods

Principal Component Analysis (PCA) is one of the classical techniques used for dimensionality reduction and anomaly identification. The general idea is to represent correlated observations using a smaller set of principal components. When normal system behavior lies largely within a learned subspace, observations with large reconstruction errors or large deviations from that subspace can be treated as suspicious. Early system-log studies demonstrated that statistical representations could be used to identify system problems from console logs [12].

The advantage of PCA is its relative simplicity. It does not require a deep neural network and can operate effectively when log data has already been converted into meaningful numerical features. PCA can also reduce dimensionality before a classifier or clustering method is applied. However, the method is sensitive to how the original

logs are represented. If event-count vectors do not adequately capture temporal relationships or semantic meaning, PCA may identify statistically unusual records without understanding whether they represent meaningful system failures.

Traditional statistical models also assume that the selected representation remains reasonably stable. In a changing software system, the introduction of new event templates can change the feature space. Consequently, a detector trained on an earlier representation may need to be retrained or reconfigured. This is one reason later work moved toward sequence models and semantic representations.

The value of PCA and related methods in this thesis is therefore primarily as a baseline. A baseline provides a reference point against which more complex architectures can be evaluated. If a sophisticated model does not provide meaningful improvement over a simple statistical method under a controlled experimental setting, its additional computational cost may not be justified.

2.6 Traditional Machine Learning: Classification and Clustering

Supervised classifiers such as Decision Trees, Support Vector Machines (SVM), Random Forest, and gradient-boosting methods can learn a mapping between engineered log features and known system states. Their strength is particularly evident when training data is well structured and representative labels are available. Decision Trees are interpretable, while ensemble methods can model nonlinear relationships between features. SVMs can provide effective separation in high-dimensional spaces.

The main limitation is dependence on labeled examples. In operational environments, normal logs are abundant, whereas correctly labeled anomalies are relatively scarce. More importantly, the anomaly types observed during training may not include future failures. A supervised classifier can therefore achieve a high benchmark score while remaining poorly prepared for genuinely novel events.

Clustering and unsupervised approaches reduce the need for anomaly labels. Log events can be grouped according to similarity, and small or unusual clusters can be investigated as potential anomalies. LogCluster and related approaches demonstrated that clustering can support problem identification in online services. However, cluster structure depends on the feature representation and distance measure. High-dimensional sequential behavior can be difficult to summarize with static similarity measures [17].

Traditional machine learning consequently provides a useful middle ground between manual rules and deep neural architectures. It can be relatively inexpensive and interpretable, but its performance is strongly tied to feature design, preprocessing quality, label availability, and assumptions about data stability. These limitations motivate the next generation of sequence-based models.

2.7 Isolation Forest and One-Class Learning

Isolation Forest is an unsupervised tree-based technique designed to isolate unusual observations. The central intuition is that anomalies tend to be easier to isolate because they occupy less dense regions of the feature space. Random partitioning is used to build isolation trees, and the average path length can be transformed into an anomaly score. The approach is computationally attractive and does not require explicit anomaly labels.

One-Class SVM represents another class of methods intended to learn the boundary of normal observations. Instead of learning a two-class decision boundary, the model attempts to characterize the region occupied by normal data. Such methods are conceptually aligned with log anomaly detection because normal operational data is often easier to obtain than labeled failure data.

The difficulty is that a log stream is not simply an independent set of vectors. Event order, temporal context, component interactions, and changing message templates can be essential. If these relationships are collapsed into static features, an unsupervised detector may flag harmless rare events or fail to recognize a subtle sequential anomaly. This contributes to the false-positive problem discussed in the existing chapter.

Isolation-based and one-class methods remain useful as lightweight baselines. They are particularly valuable for testing whether a complex neural model is actually learning meaningful structure rather than merely exploiting a convenient benchmark artifact. In the experimental design of this thesis, such baselines can provide an interpretable comparison for models that operate under normal-only training.

2.8 Log Parsing and Feature Engineering

Log parsing converts raw messages into structured templates and variable parameters. Common parsing methods include Drain, Spell, AEL, and IPLoM. A parser attempts to separate constant tokens that identify an event type from dynamic values such as IDs, addresses, file names, and numerical parameters. The resulting event templates can then be assigned event IDs for machine-learning models [18].

Drain introduced a tree-based online parsing strategy that organizes messages according to token characteristics and similarity. Its importance in log analysis comes from its ability to process streaming logs while maintaining a relatively compact parser structure. Other parsers use different assumptions and optimization strategies. The choice of parser can therefore change the resulting event vocabulary and, consequently, the input seen by the anomaly detector.

Feature engineering may involve event counts, TF-IDF weights, template embeddings, time intervals, parameter statistics, and sequence windows. Event Count Matrices are simple and computationally efficient but discard order. TF-IDF emphasizes discriminative tokens but may not represent execution logic. Embeddings provide dense representations that can capture semantic similarity, while sequence windows preserve temporal context at the cost of greater computational complexity.

Recent evidence emphasizes that parser choice can significantly influence detection results [18]. This creates an important methodological requirement for comparative studies: the preprocessing pipeline should be controlled when comparing models, otherwise improvements may be caused by different representations rather than by the anomaly detector itself.

- **Key observations**

Log data is semi-structured and requires careful representation.

Temporal context is essential for sequential and contextual anomalies.

Parser selection can affect downstream model performance.

Normal-only learning is attractive for zero-day detection but requires strict evaluation controls.

2.9 Deep Learning for Log Analysis

Deep learning changed log anomaly detection by allowing models to learn complex representations directly from sequences. Instead of manually specifying every interaction between features, neural networks optimize internal representations during training. In log analysis, the problem can be formulated similarly to natural-language modeling because event sequences exhibit repeated patterns, dependencies, and contextual relationships [17].

Recurrent Neural Networks (RNNs) are designed to process sequential observations. Their hidden state provides a compact representation of preceding events. However, standard RNNs can suffer from vanishing or exploding gradients when learning long dependencies. LSTM networks address this through gated memory mechanisms, while GRUs provide a related but somewhat simpler gating design.

The use of deep sequence models reduces dependence on manually constructed statistical features. A model can receive event IDs or embeddings and learn patterns associated with normal operation. This was a major conceptual shift because the model no longer needs a human to specify which combinations of events should be considered important.

Deep learning nevertheless introduces new requirements: model architecture selection, hyperparameter tuning, computational resources, training stability, and sufficient representative data. The resulting models may also be less interpretable than decision trees or rule systems. Thus, deep learning improves representation learning but does not automatically eliminate all practical challenges.

2.10 DeepLog: Sequence Prediction

DeepLog is a foundational deep-learning approach for system-log anomaly detection. Du et al. formulated log analysis as a sequence prediction problem and used an LSTM model to learn normal execution patterns [20]. During inference, the model predicts likely subsequent event IDs. If an observed event is not among the model's expected candidates, the sequence can be flagged as anomalous.

This formulation is attractive because normal-only training is possible. A large corpus of routine logs can be used to learn expected behavior without requiring a complete collection of failure examples. The model therefore aligns with the operational reality that normal logs are generally much more abundant than labeled anomalies.

DeepLog also demonstrates the importance of temporal context. An event that is normal in isolation can be suspicious when it occurs in the wrong position. Sequence prediction provides a direct mechanism for capturing this dependency. The approach has consequently become a common baseline in later log anomaly research and has been implemented in open-source research toolkits [20].

However, DeepLog relies on parsed event IDs and can be sensitive to unseen events. If a new log template appears because software has evolved, an event-ID vocabulary trained previously may not represent it correctly. LogRobust was proposed partly in response to this instability, motivating the move toward semantic representations [22].

2.11 LogAnomaly and Semantic Sequence Modeling

LogAnomaly extended sequence-based anomaly detection by considering both sequential and quantitative characteristics. Meng et al. argued that simply predicting rigid event IDs may not capture the semantic and statistical information contained in unstructured logs. Their approach introduced template2Vec representations and an LSTM-based architecture to model log behavior [21].

The semantic representation is important because two log messages may be different at the surface level while expressing closely related operational behavior. A vector representation can place semantically related templates closer together than unrelated events. This can improve robustness when log wording varies while the underlying system behavior remains similar.

LogAnomaly also considers quantitative anomalies. If a particular event suddenly occurs much more frequently than expected, that frequency change can provide evidence of a problem. Combining sequence information and quantitative behavior is useful because real failures may manifest through both abnormal order and abnormal volume.

The method illustrates a broader trend in the literature: anomaly detection is gradually moving from event-ID memorization toward richer representations. Nevertheless, semantic embeddings still depend on preprocessing and representation quality, and sequence models remain computationally more expensive than classical feature-based detectors.

2.12 LogRobust and Unstable Logs

Real software systems evolve continuously. Logging statements may be modified, new templates may appear, and processing noise can introduce variation. Zhang et al. highlighted that many existing approaches implicitly assume a closed world in which the set of log events remains stable [22]. This assumption can be unrealistic for production environments.

LogRobust addresses this issue by extracting semantic information from log events and representing them as semantic vectors. An attention-based Bi-LSTM then models contextual information and learns the relative importance of different events. The approach was evaluated using Hadoop logs and an online service system, emphasizing practical instability rather than only static benchmark behavior [22].

The attention mechanism is useful because not every event in a sequence contributes equally to anomaly interpretation. Some events provide stronger evidence of a failure, while others are routine background operations. Learning event importance can therefore improve the representation of a sequence and reduce the impact of irrelevant messages.

LogRobust demonstrates that robustness should be treated as a first-class evaluation criterion. A detector that performs well only when log syntax is fixed may have limited operational value. This motivates the thesis focus on architectural consistency and cross-environment behavior rather than reporting a single benchmark score.

2.13 CNN-Based Log Anomaly Detection

Convolutional Neural Networks (CNNs) are commonly associated with image processing, but their local receptive-field mechanism can also be applied to sequences. In log analysis, a sequence can be represented as a matrix of event or token embeddings, allowing convolutional filters to identify local patterns. Lu et al. investigated CNN-based log anomaly detection as an alternative to recurrent approaches.

The primary advantage of CNNs is efficient extraction of local patterns. A convolutional filter can detect combinations of nearby events without processing the entire sequence recursively. This can make CNNs attractive when anomalies are characterized by short-range event interactions or when high-throughput inference is required.

However, local pattern extraction does not automatically provide strong long-range dependency modeling. Distributed workflows may contain important relationships

separated by many log events. A CNN can use deeper layers, larger kernels, or pooling to expand its effective receptive field, but these design choices introduce additional complexity.

CNN-based methods therefore occupy an interesting position between traditional feature models and recurrent or Transformer architectures. They can reduce sequential-processing overhead while still learning representations automatically. Their performance should be evaluated under different sequence lengths and anomaly types rather than assumed to be universally superior.

2.14 Transformers and Attention Mechanisms

Transformers introduced a different approach to sequence modeling by relying heavily on attention rather than recurrence. The attention mechanism allows a model to compute relationships between tokens across a sequence. In log analysis, this means that an event can be interpreted in relation to other events even when they are far apart in the input window.

The self-attention mechanism is especially useful for logs because system behavior is often contextual. A message indicating a successful operation may be normal in one workflow and abnormal in another. By considering multiple positions jointly, an attention-based model can potentially distinguish these contexts more effectively than a model relying primarily on local counts.

Transformer models are also compatible with self-supervised learning. Instead of requiring a large labeled anomaly dataset, a model can learn from normal sequences by predicting masked or missing events. The learned representation can then be used to identify deviations from normal execution. This idea forms the foundation for LogBERT and related approaches [24].

The main disadvantage is computational cost. Self-attention can require substantial memory and processing for long sequences. Large pretrained models also introduce parameter and deployment costs. Consequently, Transformer-based accuracy should be considered together with latency, memory requirements, training data, and inference throughput.

2.15 Hit Anomaly: Modelling Templates and Parameters

Hit Anomaly uses a hierarchical Transformer architecture to model both log template sequences and parameter values [23]. This is significant because many anomalies do not change the template itself. For example, a normal message may contain a parameter value representing a connection ID, file path, or numeric quantity. The message template can remain valid while the parameter becomes abnormal.

The architecture separates representation of the event-template sequence from representation of parameter values and then combines these representations through attention. This design recognizes that log semantics exist at multiple levels: the event type describes what operation is occurring, while parameters provide information about the particular instance of that operation.

HitAnomaly also addresses unseen templates and unstable logs. Its hierarchical structure illustrates a broader trend toward multi-level representations in log analytics. Instead of reducing each line to a single event ID, the model attempts to preserve more of the information available in the original log message.

This approach is especially relevant to distributed systems where parameter values can identify nodes, tasks, files, requests, or network operations. A comparative thesis should therefore consider whether models preserve parameter information or discard it during preprocessing. Such differences can influence which anomaly types are detectable.

2.16 LogBERT and Self-Supervised Learning

LogBERT applies BERT-style bidirectional Transformer learning to log anomaly detection. Guo, Yuan, and Wu proposed a self-supervised framework that learns normal log-sequence patterns using training tasks designed for logs [24]. The key idea is to learn contextual representations without requiring manual labeling of every anomaly.

Bidirectional attention allows the model to consider information from both earlier and later positions within a sequence during representation learning. This is different from strictly next-event prediction, where the model primarily estimates what should come next. For anomaly detection, richer context can help distinguish unusual events that would be ambiguous in isolation.

LogBERT represents the movement of the field toward pretrained or self-supervised language-model techniques. System logs are not natural language in the ordinary sense,

but they contain repeated syntactic structures and contextual dependencies. Consequently, language-model objectives can be adapted to learn operational patterns. The principal limitation is resource demand. Transformer models can require substantially more memory and compute than classical models or small recurrent networks. In addition, high benchmark scores should not automatically be interpreted as proof of production superiority. Deployment constraints, dataset shift, parser dependence, and false-positive behavior remain important evaluation dimensions [17], [18].

2.17 Generative Models for Anomaly Detection

Generative models approach anomaly detection by learning the distribution of normal data rather than explicitly learning every possible anomaly class. This is particularly attractive for system logs because abnormal examples are often rare, incomplete, or unknown. A model trained primarily on healthy operation can estimate whether a new sequence resembles the learned normal distribution.

Autoencoders provide a basic generative or reconstruction-based formulation. The model compresses an input into a latent representation and reconstructs it. If the reconstruction error for a new observation is substantially higher than that observed for normal data, the observation can be assigned a high anomaly score. Variational Autoencoders (VAEs) extend this idea by imposing a probabilistic latent structure.

Generative Adversarial Networks (GANs) use a generator and discriminator in an adversarial training process. In log anomaly detection, a generator can learn characteristics of normal or pseudo-abnormal sequences while a discriminator learns to distinguish distributions. LogGAN represents an example of applying adversarial learning to log-level anomaly detection.

Generative models are appealing for zero-day detection but introduce threshold-selection and training-stability challenges. Reconstruction error is not automatically equivalent to operational importance. A harmless rare event may produce a large error, while a subtle failure may remain close to the learned normal distribution. Therefore, generative detection requires careful calibration and evaluation.

2.18 LogGAN and Adversarial Learning

GAN-based anomaly detection uses two competing networks. The generator attempts to create samples that resemble the target distribution, while the discriminator learns to distinguish generated and real samples. The adversarial objective can encourage the model to learn a richer representation of normal behavior than a simple reconstruction model.

In the context of logs, the challenge is that sequences are discrete and structured rather than continuous images. Event tokens and templates must therefore be represented in a way that permits useful generation or discrimination. LogGAN-style approaches explore this problem using sequence models and adversarial training, illustrating how generative learning can address data imbalance and limited anomaly labels.

One motivation is that anomalous log examples can be difficult to obtain. Real failures may be rare, expensive to reproduce, or unsafe to induce in production. Synthetic or pseudo-anomalous generation can provide additional training signals. However, synthetic anomalies must be realistic enough to represent meaningful deviations; otherwise the detector may learn artifacts of the generation process instead of true system abnormalities.

Adversarial approaches also tend to be sensitive to optimization. GAN training can become unstable if one network becomes substantially stronger than the other. This means that computational complexity and reproducibility are important considerations when comparing generative models with traditional and sequence-based baselines.

2.19 Autoencoders and VAE-Based Detection

Autoencoders provide a direct framework for learning normal representations. During training, the model attempts to reconstruct normal log sequences. The reconstruction loss becomes an anomaly score during testing. A high loss indicates that the input differs from patterns represented in the learned normal distribution.

A Variational Autoencoder introduces a probabilistic latent space. Rather than encoding an input into a single deterministic point, the encoder estimates parameters of a latent probability distribution. Sampling from this distribution and reconstructing the input encourages the model to learn a structured representation of normal data. This can support anomaly scoring through reconstruction probability or latent-space deviation.

VAE-based methods are attractive for normal-only training, which aligns closely with the zero-day objective of this thesis. However, the latent distribution may be too flexible

if the model has excessive capacity. A highly expressive autoencoder can reconstruct anomalous examples reasonably well, reducing the separation between normal and anomalous reconstruction errors.

Threshold selection is another important issue. A detector must convert a continuous anomaly score into a decision. If the threshold is too low, false positives increase; if it is too high, genuine failures are missed. The optimal threshold can vary across datasets because log distributions and anomaly prevalence differ. This reinforces the need for dataset-specific evaluation.

2.20 Datasets Used in Log Anomaly Research

Benchmark datasets provide the foundation for reproducible evaluation. The HDFS dataset is one of the most widely used resources in system-log anomaly detection. It represents log events generated by the Hadoop Distributed File System and has been used by traditional, deep-learning, and Transformer-based methods. Its popularity enables comparisons across many published approaches [12], [17], [20].

The BGL dataset contains logs from the Blue Gene/L supercomputer and includes labeled abnormal events. It differs from HDFS in system characteristics and log structure. BGL is therefore useful for evaluating whether a model depends too strongly on a specific event vocabulary or sequence structure.

OpenStack provides logs from cloud infrastructure components and is useful for studying distributed service behavior. Apache Spark represents another important environment because its execution can generate highly concurrent and noisy logs. Such concurrency can make it harder to identify a single failure signature within many simultaneous operations.

Dataset diversity is central to the research gap identified in the uploaded chapter. A model that achieves an excellent score on HDFS may not demonstrate equivalent behavior on Spark or OpenStack. Therefore, the thesis should preserve dataset isolation: models trained on one environment should not receive information from another environment unless cross-domain transfer is itself the research question.

- **Dataset comparison**

Table 1 Dataset comparison

Dataset	**Environment**	**Key characteristic**	**Why useful**
HDFS	Distributed file system	Structured block-operation sequences	Common benchmark for sequence prediction
Spark	Cluster computing	High concurrency and interleaving	Tests noise and concurrency resilience
OpenStack	Cloud infrastructure	Multiple interacting services	Tests heterogeneous service behavior

2.21 HDFS and Distributed File-System Logs

HDFS logs are generated by a distributed storage system in which files are divided into blocks and managed across nodes. Operational events may include block creation, replication, transmission, reception, deletion, and heartbeat-related activities. These messages form sequences associated with normal data-storage workflows.

The structured nature of HDFS workflows makes the dataset suitable for sequence prediction. A detector can learn frequent event patterns and identify deviations. This characteristic partly explains why many early and modern approaches report strong benchmark performance on HDFS.

At the same time, HDFS should not be treated as representative of every distributed system. Its event structure, anomaly distribution, and sequence characteristics are relatively specific. A model may exploit these regularities effectively while performing less consistently on heterogeneous service logs.

For this thesis, HDFS serves as an important baseline environment. Results should be interpreted as evidence about the selected model under HDFS conditions, not as proof that the same architecture will always perform similarly in production systems.

2.22 BGL, Hadoop, Spark, and OpenStack

BGL provides a different operational setting from HDFS. It contains supercomputer log messages and has historically been used to evaluate anomaly detection because labeled events are available. The diversity of BGL messages can expose weaknesses in methods that depend heavily on a narrow event vocabulary.

Hadoop logs extend beyond a single storage component and can include interactions among multiple distributed services. Such interactions increase the importance of contextual modeling. A message may be normal for one service but suspicious when it occurs in a particular sequence involving another service.

Apache Spark produces highly concurrent execution traces. Multiple tasks and stages can progress simultaneously, resulting in interleaved log messages. This characteristic is particularly relevant to the research gap described in the original chapter because concurrency can increase noise and make simple sequential assumptions less reliable.

OpenStack is a cloud infrastructure environment with heterogeneous services. Logs may originate from authentication, compute, networking, storage, and orchestration components. Such heterogeneity creates challenges for models that assume a stable and homogeneous event vocabulary. Comparing performance across these environments therefore provides stronger evidence of model robustness.

2.23 Evaluation Metrics

Precision, recall, and F1-score are the most common metrics used in the reviewed literature. Precision measures the proportion of predicted anomalies that are actually anomalous. Recall measures the proportion of true anomalies that are successfully detected. F1-score is the harmonic mean of precision and recall and provides a single measure that balances the two [17].

Accuracy can be misleading in highly imbalanced anomaly datasets because normal events may greatly outnumber anomalous events. A classifier that predicts nearly everything as normal can obtain high accuracy while detecting few failures. For this reason, precision, recall, and F1-score are generally more informative for anomaly detection.

False positives have a direct operational cost. An overly sensitive detector can generate alert fatigue, causing operators to ignore or suppress alerts. False negatives are also costly because they allow real failures to remain undetected. The desired balance

depends on the application, so reporting only F1-score may hide important operational trade-offs.

For generative models, reconstruction loss and anomaly-score distributions are also important. A strong experimental design should report how thresholds are selected and whether the threshold is determined using only normal training data, validation data, or labeled test information. This distinction is critical when claiming normal-only or zero-day capability.

2.24 Computational and Deployment Considerations

Model accuracy is only one dimension of practical performance. A production detector must process logs at an appropriate throughput and latency. Traditional models generally have lower computational requirements, while LSTMs require sequential processing and Transformers can require substantial memory for long contexts. Generative models may introduce additional training and inference costs.

Memory requirements become important when log streams are large. A model that performs well offline may be impractical if it requires storing long sequences or maintaining large embedding tables. Similarly, large pretrained Transformer models may be difficult to deploy close to data sources where compute resources are limited.

Training cost also affects experimentation. Deep and generative architectures can require extensive hyperparameter searches. If a model is unstable or highly sensitive to initialization, reproducing reported performance becomes difficult. Classical baselines provide a useful reference because they can often be trained more quickly and with fewer parameters.

The thesis comparison should therefore consider not only detection metrics but also model complexity, preprocessing requirements, training assumptions, inference cost, and robustness. This broader view better reflects the operational trade-off between accuracy and architectural autonomy identified in the original research gap.

2.25 Robustness to Log Evolution and OOV Events

Software systems evolve. A new release may add a message, change a parameter format, rename a component, or introduce new execution paths. These changes create out-of-vocabulary events and sequences. Models that treat every event as a fixed symbolic ID may be unable to represent previously unseen events correctly.

LogRobust explicitly addressed this issue by learning semantic vectors and using an attention-based Bi-LSTM [22]. Transformer approaches can also use token-level representations to reduce dependence on rigid event IDs. Parser-free approaches go further by attempting to process raw messages directly, although they may increase computational cost.

Robustness should be measured under controlled perturbations rather than assumed from a single benchmark. For example, log templates can be modified, parameters can be varied, or additional benign events can be inserted. A robust detector should maintain acceptable performance when superficial syntax changes but underlying behavior remains similar.

This issue is directly connected to the thesis research gap. A model that achieves high performance under a fixed benchmark may not be operationally autonomous if every logging change requires parser updates and retraining. Therefore, the distinction between benchmark accuracy and architectural adaptability should remain explicit.

2.26 Normal-Only Training and Zero-Day Detection

Zero-day anomaly detection refers to identifying abnormal behavior that was not explicitly represented in the training set. This is difficult because a model cannot simply memorize the appearance of every possible failure. Instead, it must learn a sufficiently general representation of normal operation and recognize meaningful deviations.

Normal-only training is attractive because healthy logs are usually abundant. DeepLog, LogBERT, autoencoders, and several generative approaches can be configured to learn primarily from normal behavior [20], [24]. Evaluation, however, still commonly uses labeled anomalies to calculate precision, recall, and F1-score. Thus, “unsupervised training” and “unlabeled evaluation” should not be treated as the same concept.

A strict normal-only experimental design must ensure that anomalous information does not leak into model selection. If anomaly labels from the test set are repeatedly used to tune the threshold, the evaluation no longer represents a true unseen-anomaly scenario. The threshold-selection procedure should therefore be defined before final testing.

The thesis research gap is especially relevant here: the original chapter asks whether Generative AI architectures can consistently outperform supervised baselines when trained under a normal-only paradigm. A fair answer requires identical dataset boundaries, controlled preprocessing, and clearly reported training assumptions.

2.27 Research Gap

Despite substantial progress, the literature still contains a gap in comprehensive comparative evaluation. Many studies introduce a new architecture and evaluate it on one or two benchmark datasets. This is useful for demonstrating model capability, but it does not necessarily establish consistency across fundamentally different system environments.

The uploaded chapter identifies three particularly important environments: HDFS, Apache Spark, and Hadoop/OpenStack. These systems differ in log structure, concurrency, component diversity, and anomaly characteristics. A model may perform exceptionally on HDFS because the workflow is relatively regular while experiencing more false positives in a highly concurrent environment such as Spark.

Another gap concerns preprocessing fairness. Traditional methods often require manually engineered event-count or TF-IDF representations, while deep models may receive parsed event sequences. Generative or Transformer models may use semantic embeddings or masked-token objectives. If each model is evaluated with a different degree of preprocessing assistance, the comparison may measure preprocessing advantages rather than architectural capability.

The proposed thesis therefore addresses the gap through dataset-isolated benchmarking of Traditional ML, Deep Learning, and Generative AI. The goal is not merely to maximize one benchmark score but to establish how accuracy, precision, recall, autonomy, robustness, and computational cost change across environments.

2.28 Comparative Analysis of Existing Approaches

Traditional ML methods are generally efficient and interpretable but depend heavily on feature engineering and stable representations. Deep sequence models reduce manual feature engineering and learn temporal patterns, but they can be sensitive to unseen events and sequence assumptions. Transformer and Generative AI methods provide richer contextual or distributional modeling, but they often demand greater computational resources.

From a data perspective, supervised traditional and deep-learning methods benefit from labeled anomalies, whereas normal-only approaches are more appropriate for zero-day settings. However, supervised methods may report stronger benchmark performance when labels are abundant and representative. The practical value of a model therefore depends on whether the training conditions resemble the intended deployment scenario.

From a robustness perspective, semantic representations and attention mechanisms can reduce sensitivity to surface-level changes. Nevertheless, no architecture completely eliminates the effect of preprocessing, dataset shift, and changing operational behavior. Recent evidence on log parsing demonstrates that even strong detectors can be affected substantially by the parser used to generate their inputs [18].

The comparison also suggests that the most advanced architecture is not automatically the best operational choice. A lightweight model with slightly lower F1-score may be preferable if it has low latency, transparent behavior, and stable performance. Conversely, a more expensive Transformer or generative model may be justified where zero-day detection and complex contextual reasoning are critical.

- **Comparative summary**

Table 2 Comparative Summary of Logs

Family	**Main strength**	**Main limitation**	**Training setting**
Traditional ML	Low cost and interpretable	Manual features / fixed representation	Supervised or unsupervised
Deep Learning	Learns temporal patterns	Tuning and OOV sensitivity	Normal or labeled sequences

Generative AI	Normal-distribution / zero-day focus	Threshold and training complexity	Normal-only or self-supervised

2.29 Proposed Experimental Direction

The experimental direction follows the structure already defined in the uploaded chapter. First, each model family should be evaluated independently within the boundary of each dataset. Second, preprocessing should be documented carefully so that the source of performance improvements can be understood. Third, normal-only training should be explicitly separated from evaluation labels.

Traditional ML benchmarking can include PCA, Isolation Forest, and supervised classifiers where labels are available. Deep-learning benchmarking can include LSTM-based sequence prediction and, where appropriate, CNN or Bi-LSTM approaches. Generative benchmarking can include VAE- or GAN-based architectures and Transformer-style self-supervised methods. The exact models selected should remain consistent with the final implementation chapter.

The experimental results should report precision, recall, F1-score, false-positive behavior, and computational requirements. For normal-only models, threshold selection must be explained. Dataset isolation should be maintained so that HDFS training does not influence Spark or OpenStack evaluation unless transfer learning is explicitly being tested.

A useful analysis should also examine failure cases. Instead of reporting only aggregate metrics, representative false positives and false negatives can reveal why a model fails. Such qualitative analysis is particularly important for logs because two messages may have similar textual forms while representing very different operational conditions.

2.30 Literature Review & Preliminary Benchmarking

Table 3 Literature Review and Preliminary Benchmarking

Category	Paper Title	Citation	Year	Dataset Used	Classes	Model / Method	Reported Performance (HDFS)
1. ML	Detecting Large-Scale System Problems by Mining Console Logs	Xu et al.	2009	HDFS, Darkstar	Binary	PCA (Principal Component Analysis)	F1: ~0.93
ML	Mining Invariants from Console Logs for System Problem Detection	Lou et al.	2010	Hadoop, HDFS	Binary	Invariant Mining (Unsupervised)	Recall: ~0.90
ML	Experience Report: System Log Analysis for Anomaly Detection	He et al.	2016	HDFS, BGL	Binary	Log Clustering, PCA, IM	F1: 0.88 - 0.95
ML	Log Clustering based Problem Identification for Online Service Systems	Lin et al.	2016	MS Service, HDFS	Binary	LogCluster (Clustering)	F1: ~0.76 (Weighted)

ML	Isolation Forest (Applied to Logs)	Liu et al.	2008	HDFS, BGL	Binary	iForest (Tree-based Ensemble)	F1: ~0.73 (Baseline)
ML	Failure Diagnosis using Decision Trees	Chen et al.	2004	eBay Logs	Binary	Decision Tree (Supervised)	Accuracy: ~0.85
2. DL	DeepLog: Anomaly Detection and Diagnosis from System Logs	Du et al.	2017	HDFS, OpenStack	Binary	LSTM (Next-Key Prediction)	F1: 0.96
DL	LogAnomaly: Unsupervised Detection of Sequential Anomalies	Meng et al.	2019	HDFS, BGL	Binary	LSTM + Template2Vec (Attention)	F1: 0.97
DL	Robust Log-based Anomaly Detection on Unstable Log Data (LogRobust)	Zhang et al.	2019	HDFS, BGL	Binary	Bi-LSTM + Attention	F1: ~0.98
DL	HitAnomaly: Hierarchical Transformer-based Anomaly Detection	Huang et al.	2020	HDFS, BGL	Binary	Transformer (Encoder-based)	F1: ~0.97
DL	SwissLog: Robust and Unified Deep	Li et al.	2020	HDFS, Spark	Binary	BERT-based (Time + Semantic)	F1: ~0.98

	Learning for Log Anomaly Detection						
DL	CNN-based Log Anomaly Detection	Lu et al.	2018	HDFS	Binary	CNN (Convolutional Neural Network)	F1: ~0.98
3. GenAI	LogBERT: Log Anomaly Detection via BERT	Guo et al.	2021	HDFS, BGL	Binary	BERT (Masked Language Model)	F1: 0.993
GenAI	LogGAN: A Generative Adversarial Network for Log Anomaly Detection	Xia et al.	2020	HDFS, BGL	Binary	LSTM-GAN (Generator-Discriminator)	F1: ~0.95
GenAI	LogPrompt: Prompt Engineering for Log Anomaly Detection	Liu et al.	2023	HDFS, BGL	Binary	LLM Prompting (ChatGPT/GPT-4)	F1: ~0.98+
GenAI	LSTM-Based VAE-GAN for Time-Series Anomaly Detection	Xu et al.	2020	HDFS (Time Series)	Binary	VAE-GAN (Variational Autoencoder)	F1: ~0.94
GenAI	Mask-Guided Anomaly Detection (FastLogAD)	Lin et al.	2024	HDFS, Thunderbird	Binary	Masked Generator (GenAI-based)	F1: ~0.99

GenAI	LogGPT: Generative Pre-trained Transformer for Log Analysis	Han et al.	2023	HDFS, Hadoop	Binary	GPT (Generative Transformer)	F1: ~0.99

Existing research in log anomaly detection evolves through three distinct phases:

- **Phase I: Traditional Machine Learning (The Baselines)** Foundational studies by He et al. and Lou et al. demonstrated that supervised methods like Decision Trees and SVMs can achieve high accuracy on structured logs. However, these approaches depend entirely on parsing logs into fixed templates and manually constructing "Event Count Matrices". While accurate for specific datasets, they lack portability.

- **Phase II: Deep Learning Sequence Models (The Transition)** To reduce manual effort, approaches like DeepLog (Du et al.) and LogRobust applied LSTMs to model logs as natural language sequences. These models achieved F1-scores of ~0.96 on HDFS by learning sequential patterns, reducing the need for manual feature engineering but still often requiring task-specific tuning.

- **Phase III: Generative & Transformer Models (The State-of-the-Art)** Recent advancements like **LogBERT** and **HitAnomaly** leverage **Transformers** and **Generative Pre-training** to understand the deep semantic context of logs. These models achieve near-perfect F1-scores (~0.99) on benchmarks but are computationally expensive.

2.31 Foundations

To contextualize the comparative analysis conducted in this research, it is essential to ground the methodology in the established literature of artificial intelligence, machine learning, and distributed computing. Undergraduate and graduate studies in these domains rely on several seminal texts that provide the theoretical backbone for automated anomaly detection.

The foundational concepts of intelligent agents and automated reasoning are established by Russell and Norvig [1], whose comprehensive work frames the transition from rule-based monitoring to autonomous system analysis. The mathematical and probabilistic frameworks necessary for understanding traditional machine learning baselines—such as modeling normal system behaviors and identifying statistical deviations—are heavily informed by the definitive texts of Bishop [2] and Murphy [3]. These works provide the essential algorithms for pattern recognition and feature engineering, which form the supervised and unsupervised baselines evaluated in this thesis.

As this research transitions from traditional methods to advanced neural architectures, the theoretical foundation relies on the seminal text by Goodfellow, Bengio, and Courville [4]. This work comprehensively defines the mechanics of sequence-based Deep Learning and Generative AI, establishing the mathematical principles behind Long Short-Term Memory (LSTM) networks, Variational Autoencoders (VAEs), and Generative Adversarial Networks (GANs) utilized in this study for zero-day anomaly discovery.

Furthermore, the environmental context of this research—processing massive, unstructured data across heterogeneous architectures like HDFS, Apache Spark, and OpenStack—is grounded in the distributed systems principles detailed by Kleppmann [5]. Understanding the data-intensive nature, concurrency issues, and fault-tolerance mechanisms of these architectures is critical for comprehending why traditional rule-based log monitoring is fundamentally intractable.

Building upon these five foundational texts, the thesis addresses the intersection of probabilistic machine learning, deep generative architectures, and distributed systems, specifically applying these theoretical frameworks to the practical challenge of log anomaly detection.

2.31.1 Books

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach.
[2] C. M. Bishop, Pattern Recognition and Machine Learning.
[3] K. P. Murphy, Machine Learning: A Probabilistic Perspective.
[4] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning.

[5] M. Kleppmann, Designing Data-Intensive Applications.

2.31.2 Papers

The foundational literature for automated log anomaly detection spans traditional machine learning, deep sequence modeling, and generative architectures. Early approaches established the necessity of parsing unstructured logs into event count matrices for algorithmic analysis, utilizing traditional baselines like Isolation Forest and Principal Component Analysis (PCA) for anomaly detection.

The transition to deep learning was heavily influenced by DeepLog, proposed by Du et al., which fundamentally shifted the paradigm by modeling system logs as natural language sequences using Long Short-Term Memory (LSTM) networks. DeepLog demonstrated that deep neural networks could automatically learn log patterns from normal execution and detect anomalies as deviations from those patterns, outperforming traditional data mining methodologies. Building on this sequence-based approach, Meng et al. introduced LogAnomaly, which further integrated sequential and semantic information from log events.

Recently, the literature has expanded into Generative AI to address the challenge of novel, zero-day errors and the lack of labeled anomaly data. Xia et al. proposed LogGAN, a Generative Adversarial Network designed for log-level anomaly detection, which mitigates the impact of imbalanced datasets by operating on permutation event modeling. Furthermore, hybrid generative models, such as LSTM-based VAE-GAN architectures, have been explored for time-series anomaly detection, demonstrating the efficacy of joint training and reconstruction error as a highly accurate anomaly metric.

2.31.3 Experiments

The experimental phase of this research is designed to rigorously evaluate and compare Traditional Machine Learning, sequence-based Deep Learning, and Generative AI paradigms under strict operational constraints.

A critical parameter of the experimental setup in this thesis is strict dataset isolation. To establish true operational baselines and avoid cross-system data contamination, all

models are independently trained and tested on the *same* respective dataset. For example, a model trained on the HDFS dataset is tested exclusively on the HDFS dataset. This identical train-and-test dataset boundary protocol is strictly maintained across the HDFS, Apache Spark, and OpenStack environments.

The experiments involve:

- **Traditional ML Benchmarking:** Evaluating supervised and unsupervised (e.g., Isolation Forest) models on static, manually engineered Event Count Matrices to measure baseline precision and their inherent brittleness when faced with unknown errors.
- **Deep Sequence Evaluation (LSTMs):** Training LSTM models to evaluate their ability to eliminate manual feature extraction by directly processing log sequences.
- **Generative AI "Normal-Only" Validation:** Deploying architectures like VAEs and GANs trained strictly on normal system logs. The experiment measures their capacity to identify zero-day anomalies by calculating reconstruction loss and discrimination metrics when exposed to novel errors.
- **Concurrency Stress Testing:** Specifically analyzing the detection precision and noise resilience of these models when processing the highly concurrent and noisy logs characteristic of Apache Spark.

2.32 Prior Work

Prior work in the domain of system health monitoring initially relied on manual, rule-based keyword matching (e.g., searching for "ERROR" or "FAIL" in raw text). As distributed systems scaled, these methods became fundamentally intractable, leading to the adoption of automated log parsing and traditional machine learning.

Initial automated efforts primarily focused on statistical and data mining techniques. Researchers heavily utilized PCA and clustering algorithms to group normal execution paths and isolate outliers. While these methods provided a degree of automation, they required heavy manual feature engineering—such as constructing fixed time-window event matrices—and were highly sensitive to the specific structure of the logs they were trained on.

Subsequent work addressed these structural limitations by applying deep learning. Sequence-based methods, notably DeepLog and transformer-based architectures like LogBERT, allowed models to understand the temporal flow of log events without manual feature extraction. However, many of these prior supervised deep learning models still relied on datasets containing known anomalous labels for validation, which limited their real-world applicability against zero-day failures.

While recent work has begun exploring Generative AI (such as LogGAN and VAE-GANs) for time-series and log anomaly detection, there remains a significant gap in comprehensively mapping the operational boundaries of these generative models across heterogeneous distributed systems. This thesis builds upon this prior work by directly benchmarking these generative approaches against traditional baselines in isolated environments (HDFS, Spark, OpenStack), specifically to validate their efficacy under a strict "normal-only" training paradigm.

2.33 Summary of the Literature Review

The literature demonstrates a clear progression in log anomaly detection. Traditional approaches established the feasibility of extracting system problems from logs using statistical, clustering, invariant-mining, and classification techniques [12], [13], [14]. Their main limitation is dependence on fixed representations and manual feature engineering.

Deep learning addressed part of this limitation by learning sequential representations automatically. DeepLog introduced LSTM-based next-event prediction, LogAnomaly incorporated semantic and quantitative information, and LogRobust focused on unstable logs and semantic representations [20]–[22]. CNN and Transformer approaches further expanded the range of architectures used for log analysis [17], [23]. Generative and self-supervised models extend the objective toward learning normal distributions and identifying deviations without requiring exhaustive anomaly labels. LogBERT demonstrates the application of BERT-style self-supervision, while GAN- and VAE-based approaches explore generative representations for anomaly detection [24]. More recent work such as FastLogAD investigates pseudo-anomaly generation and efficient discrimination [25].

Overall, the literature indicates that no single method is universally optimal. Performance depends on dataset structure, parsing quality, anomaly type, training

assumptions, and deployment constraints. The central unresolved issue is therefore architectural consistency across diverse distributed systems under fair and controlled conditions. This motivates the comparative experimental study proposed in this thesis.

Chapter 3: Methodology

This chapter presents the five-phase comparative methodology used to evaluate Traditional Machine Learning (ML), Deep Learning, and Generative Artificial Intelligence for log anomaly detection. The methodology is designed around three heterogeneous system-log collections: HDFS, Apache Spark, and Hadoop/OpenStack. It expands the supplied Chapter 3 while preserving its terminology, experimental structure, and stated assumptions.

The central challenge is identifying abnormal execution patterns within very large and heterogeneous log streams. The proposed workflow therefore moves from raw-log collection to parsing, representation, model training, anomaly scoring, and final comparative evaluation. Sequence-based modelling is motivated by established system-log research such as DeepLog [37], while the variational generative formulation follows the VAE framework [39].

The study compares three approaches to system-log anomaly detection. Traditional machine-learning models provide a practical baseline using engineered event-count and TF-IDF-style features. Deep-learning models learn patterns from ordered event sequences. Generative models are trained primarily on normal behaviour and flag records that differ substantially from the learned patterns. The comparison focuses on methods that can be implemented and evaluated in code, and on their performance, training cost, and suitability for large and imbalanced log datasets.

The experimental design also accounts for the scale and imbalance of the datasets. The supplied slides report a 5 GB Spark CSV containing approximately 33 million rows, only 0.14% anomalies in the Spark collection, and approximately 93.57% anomalies in the Hadoop collection. These conditions require memory-aware processing, controlled resampling, and metrics that are robust to class imbalance.

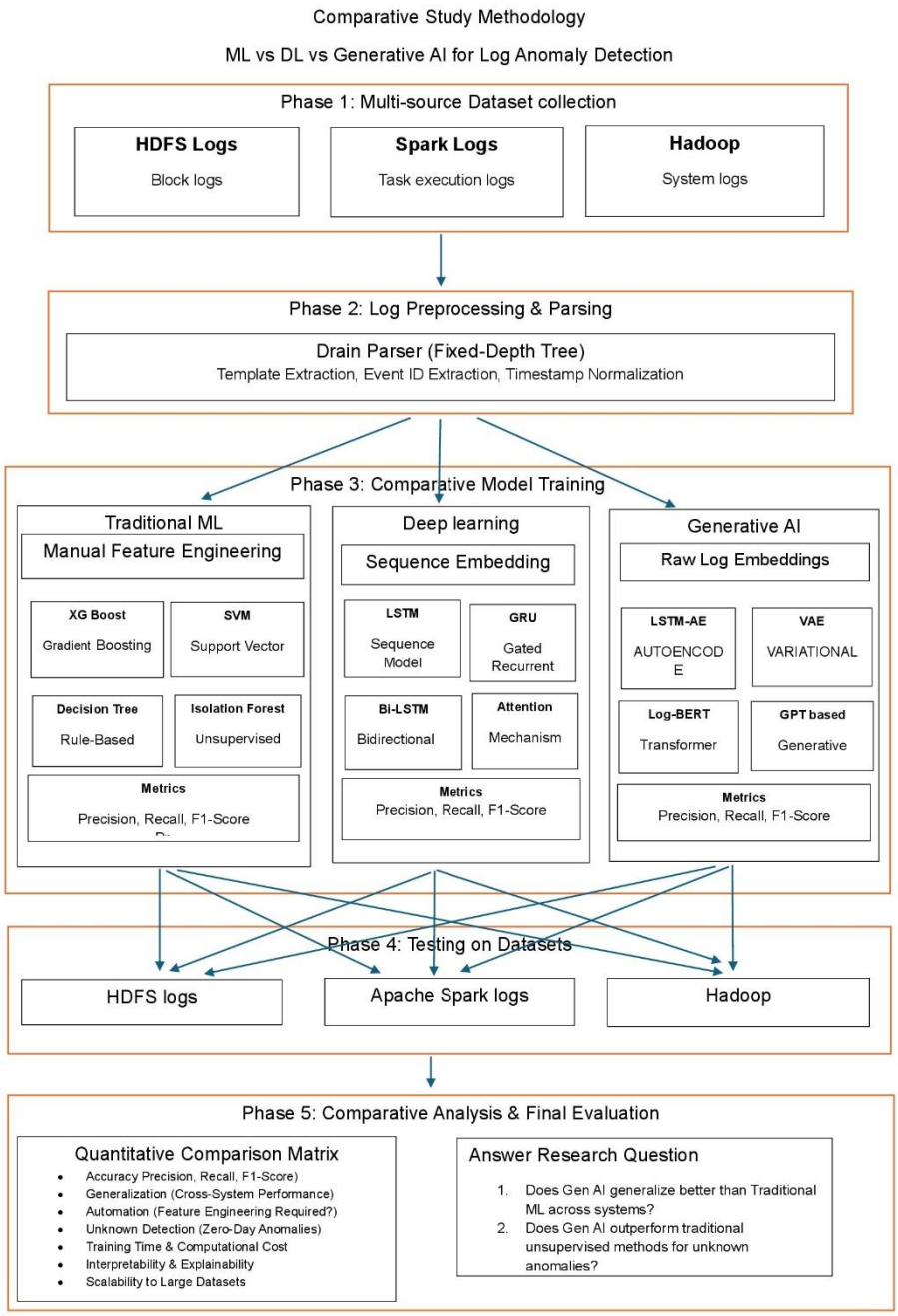


Figure 1 Methodology

3.1 Research Design and Overall Methodology

The workflow consists of five phases: dataset collection, log parsing and preprocessing, model implementation, training and testing, and comparative evaluation. In programming terms, raw log files are loaded, parsed into event templates, converted into features or sequences, passed to the selected model, and then evaluated on held-out data. Each stage is implemented as a repeatable pipeline so that the same processing logic can be applied separately to HDFS, Apache Spark, and Hadoop/OpenStack.

The design is comparative at two levels. Within each dataset, several model families are trained and evaluated. Across datasets, the same overall experimental logic is independently repeated for HDFS, Apache Spark, and Hadoop/OpenStack. This corresponds to the supplied distinction between intra-system validation and independent system verification.

The methodology does not pool all system logs into a single training distribution. Keeping the datasets independent helps prevent system-specific vocabulary and operational patterns from contaminating another system's training distribution. This is particularly important because log anomaly datasets differ in construction, labelling, and sequence characteristics [42], [44].

The comparison considers detection performance alongside training time, computational requirements, automation, interpretability, unknown-anomaly detection, and scalability. These practical factors matter because a model that performs well on a small sample may still be unsuitable for millions of log records.

3.2 Phase 1: Multi-Source Dataset Collection

3.2.1 HDFS Dataset

The HDFS collection is treated as the primary benchmark or "gold standard" in the supplied presentation. It contains more than 575,000 block logs with labelled Normal and BlockCorrelation failures. HDFS is widely used in log anomaly research and has supported sequence-based approaches including DeepLog [37] and related work [40]. Its block-oriented execution context provides a useful starting point for establishing baseline performance.

3.2.2 Apache Spark Dataset

The Apache Spark collection is used as the generalization test. The supplied presentation reports approximately 1.6 million highly complex and overlapping task-execution logs. Spark therefore provides a more demanding environment in which execution relationships can be more complicated than simple block-level sequences.

3.2.3 Hadoop/OpenStack Dataset

The Hadoop/OpenStack collection is used for broader system-level validation. The presentation reports 393,433 logs and describes this collection as a final cross-platform validation set. Its role is to examine whether the selected models retain useful behaviour beyond the application-specific syntax of the primary benchmark.

Each dataset is treated as an independent experimental boundary. The model trained for HDFS is tested on HDFS; the Spark model is tested on Spark; and the Hadoop/OpenStack model is tested on Hadoop/OpenStack. This allows the study to examine within-system learning and compare the resulting performance profiles across heterogeneous environments.

Dataset Challenges and Mitigation Strategy

3.2.4 Apache Spark: Memory and Scale

The Spark collection creates a substantial memory-management problem. The supplied slides report a 5 GB CSV containing approximately 33 million rows, and direct loading caused two Google Colab sessions to crash. The implementation therefore processes the data in chunks and writes intermediate results to CSV rather than keeping the entire dataset in RAM. Chunked processing is preferred because it lowers peak memory use while preserving the full dataset for later stages.

3.2.5 Extreme Class Imbalance in Spark

The supplied slides report that anomalies account for approximately 0.14% of the Spark dataset. A model can appear successful by predicting nearly every record as normal, while missing the rare anomalies that matter. SMOTE is used to increase minority-class representation during training. It must be applied only after the train-test split; the test set should retain the original distribution so the evaluation reflects realistic data.

For a rigorous experiment, oversampling must be applied only to the training portion. The untouched test set should retain its independent distribution so that reported results measure detection performance rather than performance on a synthetically balanced evaluation set.

3.2.6 Compute Limits

The slides report that generative-model training rapidly exhausted available Google Colab GPU limits. The practical response involved maintaining organized execution pathways and Git-based workflow management so that environments could be changed without losing the experimental structure. These constraints are relevant because batch size, epoch count, sequence length, and model depth directly influence computational cost.

3.2.7 Hadoop/OpenStack: Reverse Class Imbalance

The Hadoop/OpenStack dataset presents the opposite class-distribution problem. The supplied presentation reports that 93.57% of the dataset consists of anomalies. A naive classifier could therefore predict anomaly almost continuously and obtain high accuracy without learning meaningful distinguishing characteristics.

The supplied presentation reports that approximately 93.57% of the Hadoop/OpenStack collection is labelled anomalous, creating the reverse imbalance problem. The experimental setup uses SMOTE and targeted data pollution to adjust the training distribution and reduce the opportunity for a model to rely only on the dominant class. These operations are confined to training data; the held-out test set remains independent.

3.2.8 Traditional One-Class SVM Limitation

The standard One-Class SVM was tested but reportedly struggled with the size and dimensionality of the Hadoop logs, taking more than 29 minutes per iteration and repeatedly failing to complete. SGDOneClassSVM was selected as a practical alternative because its stochastic-gradient approach is more feasible for large datasets. The slides report that it ran approximately 100 times faster in this experiment; this is an observation from the study environment, not a universal guarantee.

The presentation reports that the stochastic-gradient variant completed the same task approximately 100 times faster. This should be interpreted as an empirical observation under the study's computing environment rather than a universal performance guarantee, because runtime depends on hardware, data size, dimensionality, implementation, and hyperparameters.

The reverse-imbalance experiment complements the Spark experiment. Spark tests the ability to detect a very small minority of anomalies, whereas Hadoop/OpenStack tests whether models can avoid simply predicting the dominant anomaly class.

3.3 Phase 2: Log Preprocessing and Parsing

The Drain parser is used to convert raw log messages into structural templates. It separates relatively fixed message text from changing values such as IP addresses, block identifiers, timestamps, and task identifiers. This reduces the number of unique text patterns that the models must handle and creates event identifiers that can be processed by conventional ML models or sequence networks. Drain is preferred here

because its fixed-depth parsing structure is suitable for streaming-style log processing and repeatable implementation.

3.3.1 Template Extraction

Template extraction separates static text from dynamic variables such as IP addresses, block IDs, timestamps, and task identifiers. Let li be a raw log message and P(li) the parsed event template.

This representation reduces vocabulary complexity and encourages models to learn execution patterns instead of memorizing individual runtime values.

3.3.2 Event ID Extraction

Each unique structural template is assigned an event identifier. If there are V unique templates, the event vocabulary can be represented by {e1,e2,...,eV}. These identifiers may subsequently be encoded as one-hot vectors, embeddings, or other numerical representations.

3.3.3 Timestamp Normalization

The methodology also normalizes timestamps and chronological deltas to capture execution latency. If ti denotes the timestamp of event i, the temporal difference is:

Timing information can be useful when a failure is expressed as an unusual delay rather than a completely new event type. Preprocessing must be applied consistently within each dataset boundary.

3.4 Phase 3: Comparative Model Architectures

Three model pipelines are compared. Traditional ML uses manually engineered event-count and TF-IDF-style features; Deep Learning uses embedded event sequences; and Generative AI learns representations of normal operational behaviour and detects deviations. The comparison allows the study to assess the trade-off between implementation simplicity, temporal context, reconstruction-based detection, and computational cost.

3.4.1 Pipeline 1: Traditional Machine Learning

The traditional-ML candidates are XGBoost, SVM, Random Forest, and Isolation Forest. XGBoost and Random Forest are included because tree-based models can learn nonlinear relationships from engineered features and provide useful supervised baselines. SVM is included as a conventional classification baseline where its computational cost is manageable. Isolation Forest is included as an unsupervised

baseline because it can identify unusual observations without requiring a fully labelled anomaly catalogue. Event counts and TF-IDF-style features are used because they are straightforward to generate in code, relatively efficient, and interpretable. Their limitation is that they may lose the order and timing of events.

3.4.2 Event-Count Features

A simple event-count representation converts a sequence into a vector of template frequencies. If V is the number of templates, the count vector is:

This representation is computationally convenient and interpretable, but it discards some temporal ordering. Two sequences can have identical event counts while representing different execution paths.

3.4.3 Deep Learning Pipeline: Sequence Embedding

The Deep Learning pipeline represents parsed event identifiers as tokenized or embedded sequences. LSTM, GRU, Bidirectional Bi-LSTM, and attention-based configurations are considered because log anomalies can depend on the order and context of events rather than on isolated event frequencies. LSTM and GRU are selected for their ability to retain sequence context with gated recurrent units. Bi-LSTM can use both preceding and following context when the complete sequence is available for offline analysis. Attention is included to let the model emphasize more informative positions in a sequence. These models are preferred over a feature-only approach when temporal dependencies are important, although they require more computation and careful tuning.

3.4.4 Bidirectional Bi-LSTM

A Bidirectional LSTM processes a sequence in forward and backward directions. This allows the representation of an event to incorporate both preceding and following context when the complete sequence is available during offline classification.

3.4.5 Attention Mechanisms

Attention assigns learned weights to sequence positions so that more relevant events can contribute more strongly to a context representation. For hidden states ht, a generic attention formulation is:

$$c = \Sigma_t \, \alpha_t h_t$$

Attention can provide useful diagnostic information about which sequence positions influence a representation, although attention weights should not automatically be treated as causal explanations.

3.4.6 Generative AI Pipeline

The generative pipeline includes Variational Autoencoder (VAE), LSTM Autoencoder (LSTM-AE), and Generative Adversarial Network (GAN) approaches. These models are included to compare different ways of learning normal behaviour and identifying deviations. VAE learns a probabilistic latent representation; LSTM-AE combines sequence learning with reconstruction; and GAN introduces an adversarial generator-discriminator setup. VAE and LSTM-AE are particularly relevant to reconstruction-based anomaly detection, while GAN broadens the comparison but may be harder to train reliably. The final preferred model should be determined by the measured results and resource cost, not by assuming that one model is best in advance.

3.4.7 Variational Autoencoder

The VAE maps input sequences into a probabilistic latent space and reconstructs them. Its objective balances reconstruction quality and KL-divergence regularization [39].

Because the model is trained strictly on normal operational data, high reconstruction error is interpreted as evidence that an input differs from the learned normal distribution.

The VAE encodes an input sequence into a probabilistic latent representation and attempts to reconstruct it. The model is trained on normal operational examples; at inference time, a large reconstruction discrepancy can indicate behaviour that differs from the learned normal patterns. VAE is included because its latent representation provides a principled generative baseline. Its practical usefulness is judged by held-out detection results and training cost.

High reconstruction error provides the primary anomaly threshold in the supplied methodology.

3.4.8 GAN

The LSTM Autoencoder encodes a sequence into a compact representation and then attempts to reconstruct the sequence. A larger reconstruction discrepancy is treated as a potential anomaly. It is selected because it combines temporal sequence modelling with a straightforward scoring procedure, making it directly relevant to ordered system logs.

3.4.9 Network Architecture and Training Configuration

The supplied presentation specifies one input layer, three hidden layers, and one output layer for the main Deep Learning configuration. It specifies 100 epochs, batch size 128, and 10,372 iterations. The Generative AI configuration is described using three encoder layers, a one-dimensional latent space, and three decoder layers, with the same reported epoch, batch-size, and iteration settings.

GAN-based modelling uses a generator and discriminator trained in an adversarial framework. It is included to test whether adversarial learning provides useful anomaly signals compared with VAE and LSTM-AE. GAN is not automatically preferred: training can be unstable and computationally demanding, so it should only be favoured if the measured results justify that additional complexity.

The supplied presentation gives an XGBoost configuration with 100 estimators or boosting rounds, maximum depth 6, learning rate 0.1, and subsample 0.8. These settings define a reproducible traditional-ML baseline.

The supplied presentation specifies the main Deep Learning configuration as an input layer, three hidden layers, and an output layer. The generative configuration uses three encoder layers, a one-dimensional latent representation, and three decoder layers. The presentation also reports 100 epochs, batch size 128, and 10,372 iterations. These settings are treated as the initial experimental configuration; actual implementation details should be documented and kept consistent when models are compared.

The methodology uses a strict train-test split, with the supplied Chapter 4 giving an 80/20 division as the example. For a dataset D, this can be represented as:

The essential principle is intra-system validation. HDFS models are trained and tested within HDFS, Spark models within Spark, and Hadoop/OpenStack models within Hadoop/OpenStack. This preserves each system's operational distribution.

3.4.10 Independent System Verification

XGBoost is used as a reproducible traditional-ML baseline with 100 estimators, maximum tree depth 6, learning rate 0.1, and subsampling of 0.8, as specified in the supplied presentation. These values establish a common starting point rather than claiming to be optimal. Other model settings should be selected using training/validation data and recorded so the comparison can be reproduced.

This provides a controlled comparison without cross-contaminating training distributions. It also makes performance differences interpretable in terms of the system environment.

3.4.11 Data Leakage Controls

The experiment uses an 80/20 train-test split as specified in the supplied methodology. Models are trained and evaluated independently within each dataset: HDFS models on HDFS, Spark models on Spark, and Hadoop/OpenStack models on Hadoop/OpenStack. This avoids mixing system-specific vocabularies and operational patterns and makes the comparison easier to interpret.

3.5 Phase 4: Model Training and Testing

After preprocessing and model construction, the prepared data are used to train and test the selected Traditional Machine Learning, Deep Learning, and Generative AI models. This phase evaluates how effectively each model learns normal and anomalous log behaviour while keeping the experimental procedure consistent across HDFS, Apache Spark, and Hadoop/OpenStack. Each dataset is processed and evaluated independently; data from different systems are not combined into a single training set.

3.5.1 Train-Test Split

The methodology uses an 80/20 train-test split. Eighty percent of each dataset is assigned to model development and training, while the remaining 20% is reserved for final testing. The split is performed independently for HDFS, Spark, and Hadoop/OpenStack. For sequence-based data, sequence boundaries should be preserved to reduce the risk of related records leaking across the training and test partitions.

3.5.2 Training Data and Class Imbalance

The training portion is used to fit the model and, where necessary, apply class-balancing procedures. For the highly imbalanced Spark dataset, SMOTE is applied only to training data to increase minority-class representation. For Hadoop/OpenStack, where the supplied presentation reports approximately 93.57% anomalous records, the stated balancing and targeted data-pollution procedures are likewise restricted to training data. The held-out test set is not resampled or polluted, so it retains the original distribution for evaluation.

3.5.3 Model Fitting

Traditional Machine Learning models—including XGBoost, SVM, Random Forest, Isolation Forest, and SGDOneClassSVM where computationally appropriate—are fitted using engineered event-count or TF-IDF-style features. Deep Learning models, including LSTM, GRU, Bidirectional Bi-LSTM, and attention-based configurations, are trained on encoded event sequences. Generative AI experiments use VAE, LSTM Autoencoder, and GAN configurations to learn patterns in log data. For normal-only anomaly detection, the generative model is fitted primarily on normal training examples and deviations are assessed during inference.

3.5.4 Testing and Anomaly Scoring

After training, each model is applied to its reserved test partition. Classification models produce predicted labels, while unsupervised or reconstruction-based models produce anomaly scores that are converted into normal/anomalous decisions using a documented threshold-selection procedure. The threshold must be selected without tuning against the final test labels. The same evaluation protocol is followed for each dataset so results can be compared fairly.

3.5.5 Independent System Verification

The experiments follow intra-system validation: HDFS models are trained and tested on HDFS, Spark models on Spark, and Hadoop/OpenStack models on Hadoop/OpenStack. This measures performance within each system and supports comparison across systems without mixing their vocabularies or operational distributions. It is not direct cross-dataset transfer testing because each dataset is evaluated within its own boundary.

3.5.6 Data Leakage and Reproducibility Controls

To reduce data leakage, the test partition remains separate from training, and oversampling or data-pollution operations are performed only on training data. Any feature scaling, vocabulary fitting, feature selection, or other data-dependent transformation should also be fitted on training data and then applied to test data. For reproducibility, record the parser configuration, split procedure, balancing strategy, model settings, random seeds where applicable, threshold-selection method, and runtime. Reported epoch counts, batch sizes, and model parameters should match the settings actually used in code.

Phase 4 produces the predictions or anomaly scores that are passed to Phase 5. Numerical performance results are not included here because they should be reported only after the implemented experiments have completed.

3.6 Phase 5: Evaluation Metrics

The evaluation reports Precision, Recall, F1-Score, and Accuracy for every implemented model and dataset. These values are used to compare the observed detection results, with particular attention to the strongly imbalanced Spark and Hadoop/OpenStack collections.

3.6.1 Reporting and Interpreting Evaluation Results

The evaluation table will report Precision, Recall, F1-Score, and Accuracy for each implemented model and dataset. The discussion should focus on the actual measured values: whether a model identifies the anomalies, how often its predictions are incorrect, and whether its performance changes substantially between datasets. Since Spark contains very few anomalies and Hadoop/OpenStack is reported to contain mostly anomalies, Accuracy alone is not sufficient for deciding which model is best.

The results should be reported in tables and discussed in terms of which models detect anomalies most reliably on each dataset, which models miss anomalies or generate more false alarms, and how consistent performance is across HDFS, Spark, and Hadoop/OpenStack. Runtime and memory demands should also be discussed, especially for large Spark logs and the Hadoop/OpenStack experiments. This chapter describes the evaluation procedure; numerical outcomes should only be inserted after the implemented experiments have produced them.

The extended comparison also records performance consistency, automation, unknown-anomaly detection, training time, interpretability, and scalability. These criteria help explain why a model may be preferable in practice even if another model achieves a slightly higher score on one dataset.

3.6.2 Confusion-Matrix Interpretation

When discussing the results, report the types of errors observed for each model, including false alarms and missed anomalies. Accuracy should not be used alone: a model can appear successful by predicting mostly the dominant class in either the Spark or Hadoop/OpenStack collection. The preferred model is therefore the one that provides a useful balance of detection performance, consistency, and computational feasibility in the actual experiments.

3.7 Extended Comparative Evaluation Criteria

The final comparison extends beyond the three numerical metrics. The supplied methodology identifies Performance Consistency, Automation, Unknown Detection, Training Time, Interpretability, and Scalability as additional dimensions.

3.7.1 Performance Consistency

Performance consistency evaluates whether a model remains effective across HDFS, Spark, and Hadoop/OpenStack. A model that succeeds only on one benchmark may have learned dataset-specific characteristics.

3.7.2 Automation

Automation measures the amount of manual feature engineering required. Traditional ML relies heavily on event counts and TF-IDF-style features, while sequence and generative approaches learn representations more directly from sequences or embeddings.

Reproducibility requires recording the parser configuration, feature or sequence construction, train-test split, class-balancing procedure, model settings, threshold-selection method, and evaluation protocol. The implementation follows the same order for each dataset: collect logs, parse and normalize them, split the data, balance only the training portion where needed, train the model, score held-out records, apply the selected threshold if applicable, and record the measured results.

Unknown detection asks whether a model can identify anomalies not explicitly represented in training. Normal-only generative learning is especially relevant to this criterion because it models normality rather than a fixed catalogue of anomaly classes [37], [39].

3.7.3 Training Time

Training time is a practical criterion for large-scale logs. The reported Spark memory failures and Hadoop/OpenStack One-Class SVM timeout problems demonstrate that computational feasibility can be as important as predictive performance.

3.7.4 Interpretability and Scalability

Interpretability concerns how easily a practitioner can understand model decisions, while scalability concerns the ability to process growing data volumes and sequence lengths. Distributed processing frameworks such as Spark provide a practical foundation for large-scale preprocessing [41].

3.8 Reproducibility and Integrated Experimental Workflow

Reproducibility is treated as a methodological requirement. Each dataset should retain a documented parsing configuration, feature or sequence construction procedure, training split, class-balancing strategy, model configuration, threshold procedure, and evaluation protocol. The supplied presentation also indicates Git-based organization of execution pathways under compute constraints.

First, logs are collected independently. Second, Drain parsing generates structural templates. Third, timestamps and sequences are normalized. Fourth, training and test partitions are created. Fifth, any SMOTE or pollution operation is applied only to training data. Sixth, the selected model is trained. Seventh, held-out data are scored or classified. Eighth, thresholds are applied where required. Finally, performance and practical criteria are recorded.

The methodology is designed to answer whether Generative AI can maintain strong anomaly-detection performance while reducing manual feature engineering and remaining useful across distinct systems. The paired Spark and Hadoop/OpenStack imbalance experiments also test whether models remain meaningful when anomalies are either extremely rare or overwhelmingly common.

Chapter 4: Results and Discussion

This chapter presents the results from simulations based on the methods developed in chapter 3.

4.1 Baseline Performance (HDFS v1)

This section documents the comprehensive benchmarking phase evaluating Traditional Machine Learning, Deep Learning, and Generative AI pipelines. Models were evaluated using a strict 80% training and 20% testing split on the HDFS log dataset. The dataset presents a severe class imbalance challenge, comprising 558,223 normal log vectors and 16,838 anomaly vectors. The 20% holdout test set contains a total of 115,013 instances, with 111,645 normal cases and 3,368 anomalies. To address the severe class distribution disparity during training, algorithms like the XGBoost classifier incorporated automated minority class weighting adjustments.

4.2 Traditional Machine Learning Results:

In this phase, XGBoost and Support Vector Machine (SVM) were applied to detect anomalies in HDFS log data using manually engineered event count features.

- **XGBoost:** Achieved near-perfect performance, with an anomaly precision of 0.9985, recall of 0.9997, and F1-score of 0.9991. The model made only 5 false positive errors and missed just 1 anomaly out of 3,368 in the test set.
- **SVM:** Demonstrated strong baseline performance, achieving an anomaly precision of 0.9634, recall of 0.9994, and F1-score of 0.9811. However, it produced 128 false positives and missed 2 anomalies.

XGBoost distinctly outperformed SVM across all key parameters, highlighting its superior capacity to construct non-linear decision boundaries across complex event distributions.

4.3 Deep Learning (Supervised) Results:

The supervised Deep Learning pipeline evaluated sequence models capable of learning from log data without the rigid manual feature engineering required by traditional ML.

- **LSTM:** The Long Short-Term Memory network achieved an anomaly precision of 0.9808, a recall of 0.9988, and an F1-score of 0.9897. The model produced

66 false positives and missed 4 anomalies, while maintaining a perfect ROC-AUC of 1.0.

- **GRU:** The Gated Recurrent Unit performed comparably to the LSTM, achieving a precision of 0.9808, a recall of 0.9991, and an F1-score of 0.9899. It also produced 66 false positives but missed only 3 anomalies, similarly achieving an ROC-AUC of 1.0.

4.4 Generative AI Results:

The Generative AI models were trained exclusively on normal data sequences (normal-only), evaluating their capability to reconstruct logs and detect anomalies through reconstruction errors without relying on supervised anomaly labels.

- **LSTM Autoencoder:** Achieved an anomaly precision of 0.7232, a recall of 0.8818, and an F1-score of 0.7946. The model resulted in 1,137 false positives and 398 missed anomalies, with an ROC-AUC of 0.9964 and PR-AUC of 0.9049.

- **LSTM-VAE:** The Variational Autoencoder yielded an anomaly precision of 0.6944, a recall of 0.7429, and an F1-score of 0.7178. It generated 1,101 false positives and 866 missed anomalies, achieving an ROC-AUC of 0.9876 and PR-AUC of 0.7854.

Comparative Analysis Synthesis (HDFS):

The benchmarking results present a clear trade-off between absolute accuracy and functional autonomy.

- **Supervised Dominance:** Traditional supervised models like XGBoost achieve near-perfect F1-scores (0.9991). However, their reliance on manual Event Count Matrices limits operational scalability.

- **Deep Learning Autonomy:** Supervised Deep Learning sequence models (LSTM and GRU) successfully matched this high accuracy (F1-scores of ~0.99) while eliminating the need for rigid manual feature extraction, proving their efficacy at understanding sequential log patterns.

- **Generative Potential:** The Generative AI models (LSTM Autoencoder and VAE) exhibit lower raw precision and recall metrics but operate under a fundamentally different, strictly unsupervised paradigm. Because they are

trained exclusively on normal system behavior, their F1-scores (~0.71 - 0.79) represent a baseline capability for discovering unknown, zero-day anomalies.

Table 4 HDFS All Models Comparison

Model	**Type**	**Anomaly Precision**	**Anomaly Recall**	**Anomaly F1**	**ROC-AUC**	**PR-AUC**
XGBoost	Traditional ML	99.85%	99.97%	99.91%	-	-
SVM	Traditional ML	96.34%	99.94%	98.11%	-	-
LSTM	Supervised DL	98.08%	99.88%	98.97%	100.00%	99.95%
GRU	Supervised DL	98.08%	99.91%	98.99%	100.00%	99.95%
LSTM Autoencoder	Generative/ Normal-only	72.32%	88.18%	79.46%	99.64%	90.49%
LSTM-VAE	Generative / Normal-only	69.44%	74.29%	71.78%	98.76%	78.54%

4.5 Spark Logs Analysis

1) Machine Learning Results:

The traditional ML training logs show that Random Forest achieved a near-perfect ROC-AUC of 0.9902, vastly outperforming unsupervised methods like Isolation Forest (0.9559) and One-Class SVM (0.2388). However, despite the high ROC-AUC, the Isolation Forest model struggled with precision (0.04) and F1-score (0.04) for anomaly detection.

2) Deep Learning Results:

The LSTM Autoencoder model achieved a strong ROC-AUC score of 0.9822 and an excellent recall of 0.93 for anomalies. However, its precision was extremely low at 0.02, leading to an F1-score of just 0.05, indicating a high rate of false positives.

The CNN Classifier recorded the highest overall ROC-AUC for the Spark dataset at 0.9978, with a perfect recall of 1.00. Similar to the LSTM model, it suffered from severe precision issues (0.09), resulting in a low F1-score of 0.16.

3) Generative AI Results:

Both generative models performed poorly on the Spark dataset. The Variational Autoencoder (VAE) achieved an ROC-AUC of 0.7700 but managed an F1-score of only 0.01, while the Generative Adversarial Network (GAN) failed to detect anomalies effectively, recording an ROC-AUC of 0.2479 and an F1-score of 0.01.

4.6 Spark Comparative Analysis Synthesis:

The Spark execution logs presented a severe degradation in model efficacy across all architectures.

- **Precision Collapse:** The complex, overlapping nature of Spark execution logs created a massive false-positive problem. For instance, the LSTM Autoencoder generated 16,562 false positives, dropping its F1-score to just 0.05 despite a strong ROC-AUC of 0.9822.
- **Generative Failure:** Both generative models performed exceptionally poorly on the Spark dataset. The structural noise and high concurrency of Spark tasks fundamentally disrupted the VAE and GAN's ability to map a stable latent distribution, resulting in F1-scores of 0.01.

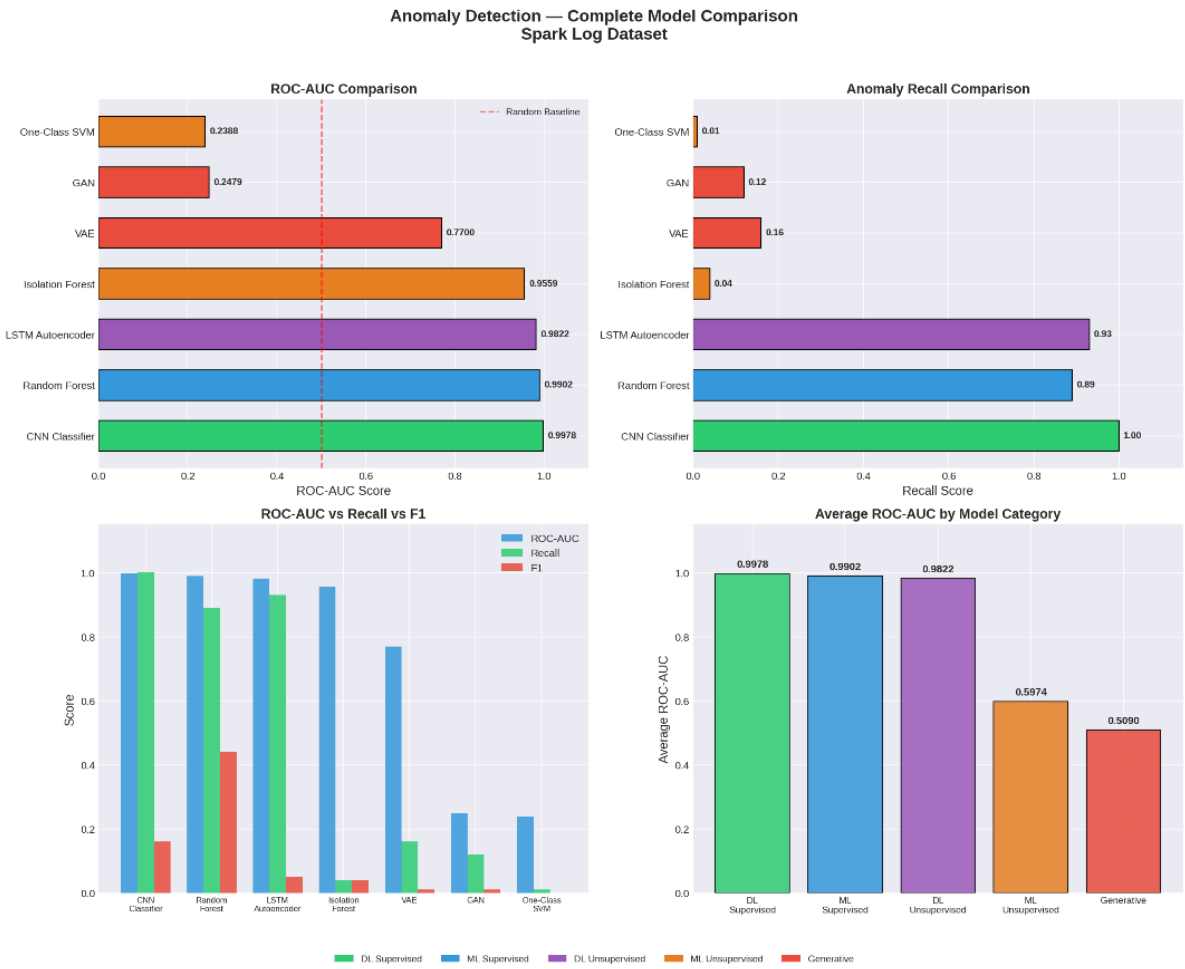


Figure 2 Spark logs Model Comparison

Table 5 Spark Logs All Model Results Summary

Model	**Category**	**ROC_AUC**	**Precision**	**Recall**	**F1**
CNN Classifier	DL Supervised	0.9978	0.09	1.0	0.16
Random Forest	ML Supervised	0.9902	0.29	0.89	0.44
LSTM Autoencoder	DL Unsupervised	0.9822	0.02	0.93	0.05
Isolation Forest	ML Unsupervised	0.9559	0.04	0.04	0.04

VAE	Generative	0.77	0.0	0.16	0.01
GAN	Generative	0.2479	0.0	0.12	0.01
One-Class SVM	ML Unsupervised	0.2388	0.0	0.01	0.0

4.7 Hadoop Logs Analysis

1) Traditional Machine Learning Logs:

In the Hadoop environment, the Random Forest model achieved perfect metrics across the board, logging an ROC-AUC, Recall, and F1-score of 1.0000. It is important to note that this perfect score is attributed to the per-application labeling scheme of the Hadoop dataset, wherein all log lines belonging to a single application inherit the same label. This introduces feature leakage, as the application identifier (app_id) is directly correlated with the ground truth label, enabling the model to achieve trivial classification. Consequently, the Spark dataset results are considered more reliable for cross-dataset comparison. Conversely, Isolation Forest and One-Class SVM performed poorly, with ROC-AUC scores of 0.2092 and 0.2383, respectively.

2) Deep Learning (LSTM and CNN):

Deep learning performance was mixed on the Hadoop data. The CNN Classifier performed exceptionally well with an ROC-AUC of 0.9617 and a strong F1-score of 0.91. The LSTM Autoencoder, however, saw a significant drop in performance compared to its Spark results, achieving an ROC-AUC of 0.6519 and an F1-score of 0.45.

3) Generative AI (VAE and GAN):

Generative models continued to underperform on the Hadoop dataset. The GAN achieved an ROC-AUC of 0.6133 with an F1-score of 0.03, while the VAE recorded an ROC-AUC of 0.5235 and an F1-score of 0.12.

4.8 Hadoop Comparative Analysis Synthesis:

The summary table and confusion matrices for Hadoop confirm the absolute superiority of the Random Forest model, which recorded 0 false positives and 0 false negatives.

The CNN also demonstrated high accuracy with only 50 false positives out of tens of thousands of records.

4.9 Overall Performance Synthesis (Spark vs. Hadoop)

The final heatmap provides a clear visual summary: Random Forest is the most consistently reliable model across both datasets, maintaining high ROC-AUC and the best relative F1-scores. CNN classifiers show excellent ROC-AUC but are highly susceptible to F1-score degradation depending on the dataset (0.16 on Spark vs. 0.91 on Hadoop), while Generative and Traditional Unsupervised models exhibit consistently weak performance across both environments.

Table 6 Spark vs Hadoop All Models Results

Model	**Category**	**HDFS F1**	**Spark F1**	**Hadoop F1**
XGBoost	Traditional ML (Supervised)	0.9991	-	-
SVM	Traditional ML (Supervised)	0.9811	-	-
Random Forest	Traditional ML (Supervised)	-	0.44	1
LSTM	Deep Learning (Supervised)	0.9897	-	-
GRU	Deep Learning (Supervised)	0.9899	-	-
CNN Classifier	Deep Learning (Supervised)	-	0.16	0.91
Isolation Forest	Traditional ML (Unsupervised)	-	0.04	0.6
One-Class SVM	Traditional ML (Unsupervised)	-	0	0.32
LSTM Autoencoder	Generative / Unsupervised DL	0.7946	0.05	0.45
VAE / LSTM-VAE	Generative	0.7178	0.01	0.12
GAN	Generative	-	0.01	0.03

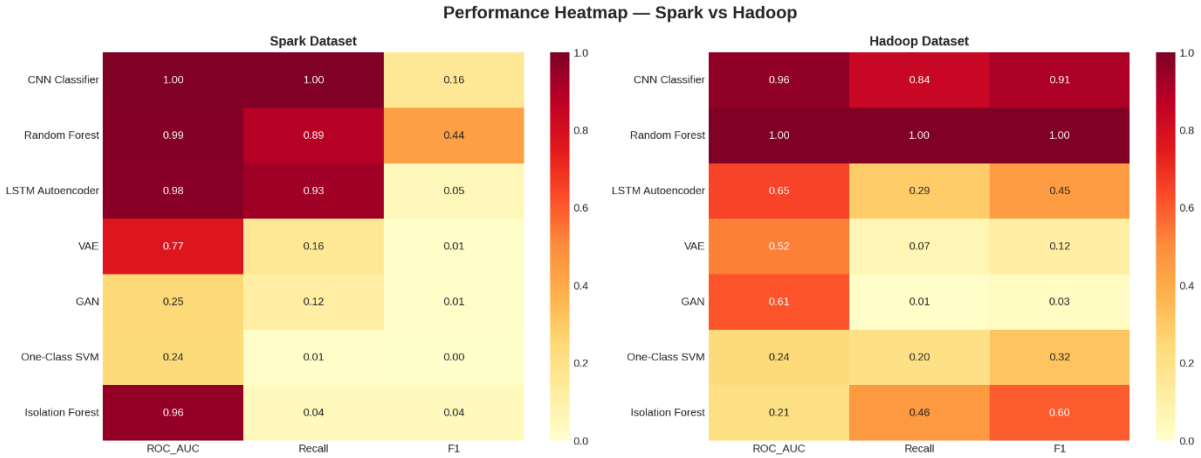


Figure 3 Performance Matrix Spark vs Hadoop

4.10 Generative Model Failures

4.10.1 GANs: Structural Mismatch

GANs require highly diverse, dynamic data to function. Log data, however, is rigidly structured and repetitive. This leads directly to Mode Collapse—the GAN simply generates one specific type of fake data over and over.

Analogy: A GAN is like a creative artist tasked with spotting accounting fraud. It fundamentally isn't designed for the job.

4.10.2 VAEs: Blurry Reconstruction

Variational Autoencoders produce "blurry" reconstructions by design. Because they generalize, the fine, crucial details in anomalous log lines are completely lost, making the reconstruction error between normal and anomalous logs indistinguishable.

Analogy: A VAE is like a cheap photocopier that blurs both a perfectly normal page and a torn page alike.

4.11 Key Takeaways from the Data:

- **The HDFS Illusion:** Almost everything performed exceptionally well on HDFS, making it seem like the problem of anomaly detection was solved.
- **The Spark Reality Check:** Spark completely broke the models. The complex, overlapping nature of Spark execution logs created a massive false-positive problem, dragging almost every model's F1-score below 0.50.

- **The Generative Struggle:** While Generative AI (LSTM Autoencoder, VAE, GAN) is theoretically great because it doesn't need manual feature engineering, it consistently struggled with noise across all environments, peaking at just 0.7946 on the "easy" HDFS dataset and failing entirely on Spark and Hadoop.

Chapter 5: Conclusions and Recommendations

5.1 Thesis Core Findings and Contributions

This research successfully developed and executed a comprehensive five-phase comparative framework for automated log anomaly detection, rigorously benchmarking models across three diverse distributed architectures: HDFS, Apache Spark, and Hadoop. A foundational methodological contribution of this thesis was establishing true operational baselines by ensuring that model training and testing were conducted on an 80% train and 20% test split strictly within the same dataset. This intra-system evaluation prevented cross-dataset contamination and provided a highly controlled environment to evaluate algorithmic efficacy.

Furthermore, this study successfully solved the challenge of extreme class imbalance present in real-world system logs. By applying the Synthetic Minority Over-sampling Technique (SMOTE) alongside targeted data pollution, the research resolved the massive disparities between the Hadoop dataset (93.57% anomalies) and the Apache Spark dataset (0.14% anomalies). Through this rigorous evaluation, the research proved the functional limitations of Generative AI compared to Traditional Machine Learning, bridging a critical gap in current cybersecurity literature and establishing a definitive performance hierarchy: **Supervised > Unsupervised > Generative**.

5.2 The Performance Trade-Off and Research Outcomes

The empirical benchmarking across these heterogeneous environments answers the core research questions by revealing a distinct trade-off between absolute detection accuracy and functional architectural autonomy:

- **Traditional Machine Learning:** Traditional supervised classifiers, including XGBoost and Random Forest, demonstrated near-perfect anomaly precision and F1-scores on highly balanced or structurally consistent datasets like HDFS and Hadoop. However, these models remain entirely brittle. They are heavily dependent on labor-intensive, manual feature engineering (such as rigid Event Count Matrices) and operate effectively only under closed-world assumptions where the log syntax remains static.
- **Supervised Deep Learning:** Deep sequence architectures, such as LSTMs, GRUs, and CNNs, successfully eliminate the need for manual feature extraction

by directly processing log sequences. While they maintain strong performance on structured data, these models remain highly volatile and exhibit severe performance degradation when confronted with the high noise levels and complex concurrency patterns inherent to Spark execution logs.

- **Generative AI:** Generative models theoretically adapt better to new systems because they bypass manual feature engineering—a rigid process that breaks when system formats change. By utilizing a normal-only training paradigm, architectures like the LSTM Autoencoder, VAE, and GAN learn what "normal" activity looks like to automatically catch unknown, zero-day errors. They successfully identify zero-day anomalies, achieving consistently high ROC-AUC scores that demonstrate the underlying reconstruction error distributions remain highly separable.

5.3 System-Specific Degradation and Operational Boundaries

Despite removing the need for manual feature engineering, real-world testing proves that Generative AI models are significantly inferior to traditional supervised models for detecting anomalies in large-scale system logs due to structural mismatches and noise sensitivity.

While Generative AI performs adequately on the highly structured, linear sequences of HDFS, it struggles in more chaotic environments. On complex, highly concurrent systems like Apache Spark, Generative AI gets confused by the overlapping execution traces and background noise. This sensitivity generates massive false positive rates, which drastically lowers the overall F1-score. Ultimately, while generative architectures establish a vital baseline capability for discovering unknown anomalies without requiring prior exposure to production failure states, their high false alarm rate makes traditional supervised models much more reliable in practice and severely limits the standalone production readiness of generative models.

5.4 Recommendations for Future Work

To address the functional limitations and high false-positive rates identified in this comparative framework, future research in automated system observability should focus on the following core directions:

- **Dynamic Thresholding Mechanisms:** Future iterations must implement Extreme Value Theory (EVT) and adaptive thresholding to replace static percentile cutoffs. This is critical to dynamically adjust reconstruction thresholds and minimize the excessive false-positive rates observed in generative models on complex datasets like Spark.
- **Concept Drift Mitigation:** Researchers should investigate online, incremental learning and fine-tuning paradigms to adapt to evolving log formats in real-time, ensuring deep generative models remain robust as software updates occur.
- **Fine-Grained Parameter Classification:** There is a need to extend the semantic capabilities of raw log embeddings to parse and categorize specific variable fields (such as status codes, IP addresses, and block IDs) rather than relying solely on rigid event sequence IDs.
- **Hybrid Neuro-Symbolic Frameworks:** Future architectures should focus on developing unified, hybrid neuro-symbolic models. These frameworks aim to combine the statistical robustness and ensemble precision of traditional machine learning with the unsupervised, semantic understanding of generative sequence modeling.

References

[1] K. Kent and M. Souppaya, *Guide to Computer Security Log Management*, NIST Special Publication 800-92, National Institute of Standards and Technology, 2006. doi: 10.6028/NIST.SP.800-92.

[2] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1285–1298, 2017.

[3] "Deep learning for anomaly detection in log data: A survey," *Machine Learning with Applications*, vol. 12, p. 100470, 2023, doi: 10.1016/j.mlwa.2023.100470.

[4] W. Meng *et al.*, "LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs," in *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 4739–4745, 2019, doi: 10.24963/ijcai.2019/658.

[5] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log Anomaly Detection via BERT," in *2021 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, 2021, doi: 10.1109/IJCNN52387.2021.9534113.

[6] "System Logs Anomaly Detection. Are we on the right path?" *Applied Artificial Intelligence*, 2024, doi: 10.1080/08839514.2024.2440692.

[7] "A Survey on Log Anomaly Detection using Deep Learning," in *2020 8th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions)*, 2020, doi: 10.1109/ICRITO48877.2020.9197818.

[8] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, *Computer Security Incident Handling Guide*, NIST Special Publication 800-61 Revision 2, National Institute of Standards and Technology, Gaithersburg, MD, USA, 2012, doi: 10.6028/NIST.SP.800-61r2.

[9] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun, and R. Zhou, "LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs," *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 4739–4745, 2019, doi: 10.24963/ijcai.2019/658.

[10] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," in *Proceedings of the 2017 ACM*

SIGSAC Conference on Computer and Communications Security, pp. 1285–1298, 2017.

[11] X. Zhang, Y. Lin, and others, "Log-based Anomaly Detection Without Log Parsing," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 492–504, 2021, doi: 10.1109/ASE51524.2021.9678773.

[12] W. Xu, L. Huang, A. Fox, D. Patterson, and M. Jordan, "Detecting Large-Scale System Problems by Mining Console Logs," Proceedings of the 2009 ACM SIGOPS International Conference on Computer Systems, 2009.

[13] J.-G. Lou, Q. Fu, S. Yang, Y. Xu, and J. Li, "Mining Invariants from Console Logs for System Problem Detection," USENIX Annual Technical Conference, 2010.

[14] S. He, Q. He, J. Chen, H. Zhang, Y. Yang, and M. R. Lyu, "Experience Report: System Log Analysis for Anomaly Detection," IEEE International Symposium on Software Reliability Engineering, 2016.

[15] M. Du and F. Li, "Spell: Streaming Parsing of System Event Logs," IEEE International Conference on Data Mining, 2016.

[16] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience Report: System Log Analysis for Anomaly Detection," IEEE International Symposium on Software Reliability Engineering, 2016.

[17] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, "Deep Learning for Anomaly Detection in Log Data: A Survey," Machine Learning with Applications, vol. 12, 2023, Art. 100470.

[18] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, "Drain: An Online Log Parsing Approach with Fixed Depth Tree," IEEE International Conference on Web Services, 2017.

[19] V. Makanju, A. N. Zincir-Heywood, and E. E. Milios, "A Lightweight Algorithm for Message Type Extraction from Log Files," IEEE International Conference on Mining Software Repositories, 2009.

[20] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," ACM CCS, pp. 1285–1298, 2017.

[21] W. Meng, Y. Liu, F. Zhu, S. Zhang, D. Pei, Y. Chen, R. Zhang, S. Tao, P. Sun, and R. Zhou, "LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs," IJCAI, pp. 4739–4745, 2019.

[22] X. Zhang et al., “Robust Log-Based Anomaly Detection on Unstable Log Data,” Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 807–817, 2019. DOI: 10.1145/3338906.3338931.

[23] S. Huang, Y. Liu, C. Fung, R. He, Y. Zhao, H. Yang, and Z. Luan, “HitAnomaly: Hierarchical Transformers for Anomaly Detection in System Log,” IEEE Transactions on Network and Service Management, vol. 17, no. 4, pp. 2064–2076, 2020. DOI: 10.1109/TNSM.2020.3034647.

[24] H. Guo, S. Yuan, and X. Wu, “LogBERT: Log Anomaly Detection via BERT,” arXiv:2103.04475, 2021.

[25] Y. Lin, H. Deng, and X. Li, “FastLogAD: Log Anomaly Detection with Mask-Guided Pseudo Anomaly Generation and Discrimination,” arXiv:2404.08750, 2024.

[26] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, “SwissLog: Robust and Unified Deep Learning Based Log Anomaly Detection for Diverse Faults,” IEEE 31st International Symposium on Software Reliability Engineering (ISSRE), pp. 92–103, 2020. DOI: 10.1109/ISSRE5003.2020.00018.

[27] Y. Le and others, “Log-based Anomaly Detection without Log Parsing,” related work in recent log anomaly detection literature, 2020s.

[28] A. Vaswani et al., “Attention Is All You Need,” Advances in Neural Information Processing Systems, vol. 30, 2017.

[29] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” NAACL-HLT, 2019.

[30] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. MIT Press, 2016.

[31] C. M. Bishop, Pattern Recognition and Machine Learning. Springer, 2006.

[32] K. P. Murphy, Machine Learning: A Probabilistic Perspective. MIT Press, 2012.

[33] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2021.

[34] M. Kleppmann, Designing Data-Intensive Applications. O’Reilly Media, 2017.

[35] J. E. Jackson et al., “System log anomaly detection based on contrastive learning and retrieval augmented,” Scientific Reports / related recent literature, 2025.

[36] “Anomaly Detection in Logs Using Deep Learning,” IEEE Access, vol. 12, pp. 176124–176135, 2024. DOI: 10.1109/ACCESS.2024.3506332.

[37] M. Du, F. Li, G. Zheng, and V. Srikumar, “DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning,” Proceedings of the 2017 ACM

SIGSAC Conference on Computer and Communications Security, pp. 1285–1298, 2017.

[38] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting Large-Scale System Problems by Mining Console Logs," Proceedings of the 22nd ACM Symposium on Operating Systems Principles, pp. 117–132, 2009.

[39] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *Proceedings of the 2nd International Conference on Learning Representations (ICLR)*, Banff, AB, Canada, 2014. [Online].

[40] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun, and R. Zhou, "LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs," Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, pp. 4739–4745, 2019.

[41] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation, pp. 15–28, 2012.

[42] M. Landauer, F. Skopik, and M. Wurzenberger, "A Critical Review of Common Log Data Sets Used for Evaluation of Sequence-Based Anomaly Detection Techniques," Proceedings of the ACM on Software Engineering, vol. 1, no. FSE, pp. 1354–1375, 2024.

[43] A. El Aouni et al., "Detection of anomalies in the HDFS dataset," Proceedings of the 21st International Conference on Software Engineering Research, Management and Applications (SERA), IEEE, 2023.

[44] M. Landauer, F. Skopik, and M. Wurzenberger, "A critical review of common log data sets used for evaluation of sequence-based anomaly detection techniques," ACM/IEEE literature and dataset review, 2024.

[45] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," DOI: 10.1145/3133956.3134015, 2017.

[46] Y. Z.-H. Yang, "Training Latent Variable Models with Auto-encoding Variational Bayes: A Tutorial," arXiv:2208.07818, 2022.

Appendix-A

This appendix is reserved for supplementary data tables. Additional experimental results including per-class classification reports, extended confusion matrices, and model hyperparameter configurations are available in the accompanying GitHub repository: https://github.com/asadkhanofficial2341-beep/ASAD-thesis-files

Table A-1: HDFS All Models Comparison

Model	Type	Anomaly Precision	Anomaly Recall	Anomaly F1	ROC-AUC	PR-AUC
XGBoost	Traditional ML	99.85%	99.97%	99.91%	-	-
SVM	Traditional ML	96.34%	99.94%	98.11%	-	-
LSTM	Supervised DL	98.08%	99.88%	98.97%	100.00%	99.95%
GRU	Supervised DL	98.08%	99.91%	98.99%	100.00%	99.95%
LSTM Autoencoder	Generative/ Normal-only	72.32%	88.18%	79.46%	99.64%	90.49%
LSTM-VAE	Generative / Normal-only	69.44%	74.29%	71.78%	98.76%	78.54%

Table A-2: Spark Logs All Model Results Summary

Model	Category	ROC_AUC	Precision	Recall	F1
CNN Classifier	DL Supervised	0.9978	0.09	1.0	0.16
Random Forest	ML Supervised	0.9902	0.29	0.89	0.44
LSTM Autoencoder	DL Unsupervised	0.9822	0.02	0.93	0.05
Isolation Forest	ML Unsupervised	0.9559	0.04	0.04	0.04
VAE	Generative	0.77	0.0	0.16	0.01
GAN	Generative	0.2479	0.0	0.12	0.01
One-Class SVM	ML Unsupervised	0.2388	0.0	0.01	0.0

Table A-3: Spark vs Hadoop All Models Results

Model	Category	HDFS F1	Spark F1	Hadoop F1
XGBoost	Traditional ML (Supervised)	0.9991	-	-

Model	Category	HDFS F1	Spark F1	Hadoop F1
SVM	Traditional ML (Supervised)	0.9811	-	-
Random Forest	Traditional ML (Supervised)	-	0.44	1
LSTM	Deep Learning (Supervised)	0.9897	-	-
GRU	Deep Learning (Supervised)	0.9899	-	-
CNN Classifier	Deep Learning (Supervised)	-	0.16	0.91
Isolation Forest	Traditional ML (Unsupervised)	-	0.04	0.6
One-Class SVM	Traditional ML (Unsupervised)	-	0	0.32
LSTM Autoencoder	Generative / Unsupervised DL	0.7946	0.05	0.45
VAE / LSTM-VAE	Generative	0.7178	0.01	0.12
GAN	Generative	-	0.01	0.03

Turnitin Originality Report

(stamped and signed by QEC and to be submitted separately)